

WHATNEXT VISION MOTORS - SALESFORCE CRM

Submitted in partial fulfillment of the requirements for the
Naan Mudhalvan / Smart Bridge - Salesforce Developer Virtual Internship

College Name: KSK COLLEGE OF ENGINEERING AND TECHNOLOGY

Team ID: SWTID-2026-1309

S.No	Student Name	Register Number	NM ID
1	S. PRASANNA LOGANATHAN	82102343022	4A7C7B9F6AAF3CD EB47F197765D2A8
2	V. KAVIN SHARMA	82102343013	3BE91FC3A45FCE0B 02F45D58FAC35467
3	M.S. ABSAR JEELAN	82102343002	18AD974CDDBBA2E8F B04DC7CB0DEF05

TABLE OF CONTENT

S.No.	CONTENT	PAGE NO
1	INTRODUCTION & USER STORY	3
2	IDEATION PHASE	4
	2.1 Problem Statement	4
	2.2 Empathy Map Canvas	5
	2.3 Brainstorming & Idea Prioritization	6
3	REQUIREMENT ANALYSIS PHASE	7
	3.1 Customer Journey Map	7
	3.2 Solution Requirements	8
	3.3 Data Flow Diagram	9
	3.4 Technology Stack	10
4	PROJECT DESIGN PHASE	11
	4.1 Problem-Solution Fit	11
	4.2 Proposed Solution	12
5	SPRINT & PRODUCT PLANNING	13
	5.2 Product Backlog and Sprint Schedule	14
6	PROJECT DEVELOPMENT PHASE	16
	Milestone 2: Data Management - Objects	19
	Milestone 4: Data Management - App Manager	24
	Milestone 5: Data Management - Fields	26
	Milestone 6: Record-Triggered Flows & Automation	30
	Milestone 7: Apex and Batch Class	44
9	CONCLUSION	60

WHATNEXT VISION MOTORS: SHAPING THE FUTURE OF MOBILITY WITH INNOVATION AND EXCELLENCE

User Story:

WhatNext Vision Motors implemented a Salesforce-based CRM solution to modernize vehicle ordering and dealership operations. The system automates vehicle order placement, validates stock availability, assigns orders to the nearest dealer, schedules test drive reminders, and updates order statuses dynamically. This transformation replaces manual and disconnected processes with a centralized, automated platform, improving operational efficiency and customer satisfaction.

1. INTRODUCTION

1.1 Project Overview:

WhatNext Vision Motors implemented a Salesforce-based CRM solution to modernize vehicle ordering and dealership operations. The system automates vehicle order placement, validates stock availability, assigns orders to the nearest dealer, schedules test drive reminders, and updates order statuses dynamically. This transformation replaces manual and disconnected processes with a centralized, automated platform, improving operational efficiency and customer satisfaction.

1.2 Purpose:

The purpose of this project is to automate vehicle ordering, stock management, dealer assignment, and customer engagement using Salesforce CRM. The system aims to reduce manual intervention, eliminate order errors, provide real-time tracking, and enhance overall business productivity while delivering seamless customer experience.

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

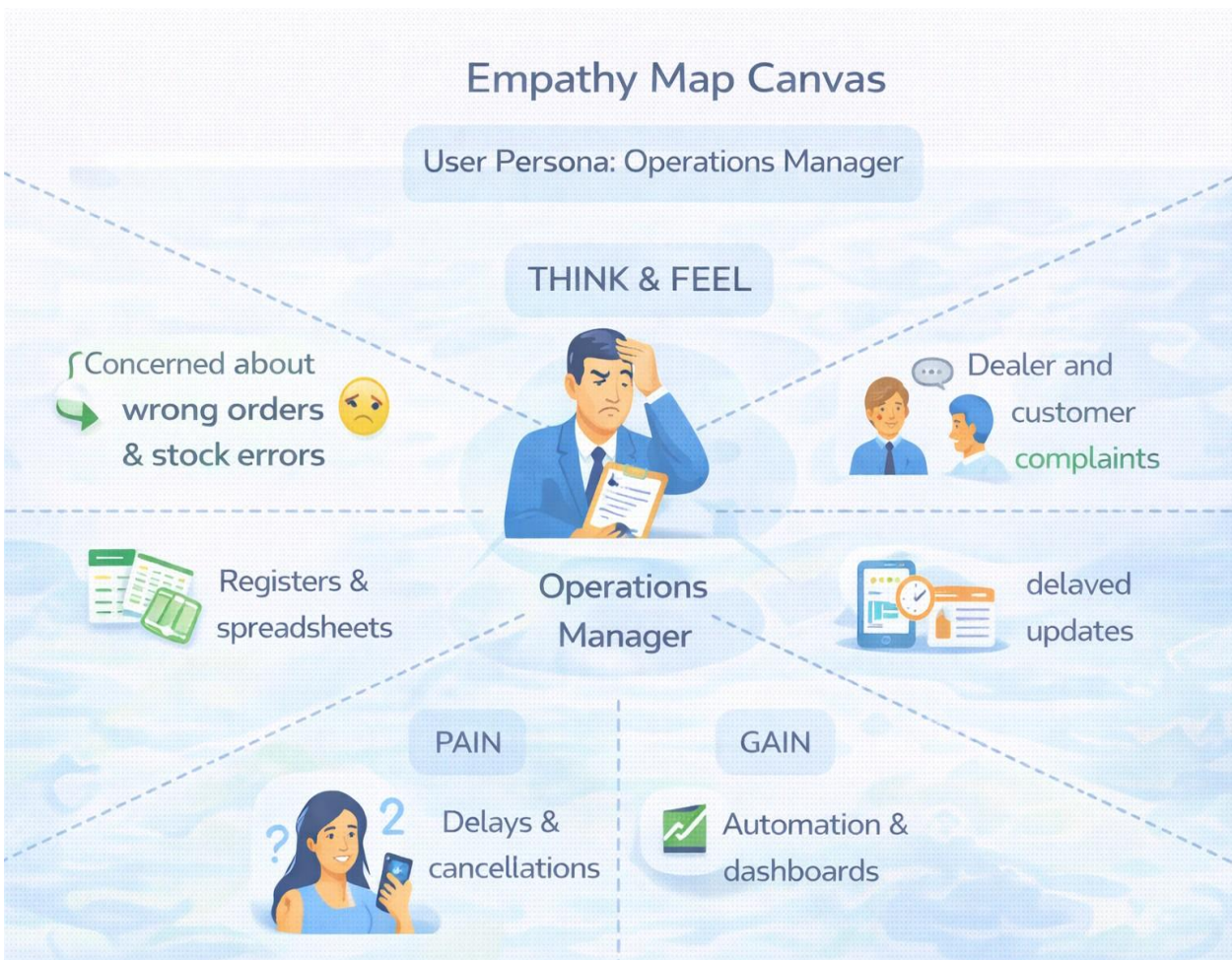
PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Operations Manager	Manage vehicle orders	Stock info is manual	Systems are disconnected	Stressed
PS-2	Customer	Place vehicle orders	Availability is unclear	Updates are delayed	Uncertain

PS	I am	I'm trying to	But	Because	Which makes me feel
PS-1	 Operations Manager	manage vehicle orders	stock info is manual	 systems are disconnected	 stressed
PS-2	 Customer	place vehicle orders	availability unclear	 delayed updates	 uncertain

2.2 Empathy Map Canvas

User Persona: Operations Manager

- **Think & Feel:** Concerned about wrong orders & stock errors
- **Hear:** Dealer and customer complaints
- **See:** Registers & spreadsheets
- **Say & Do:** Manual follow-ups
- **Pain:** Delays & cancellations
- **Gain:** Automation & dashboards



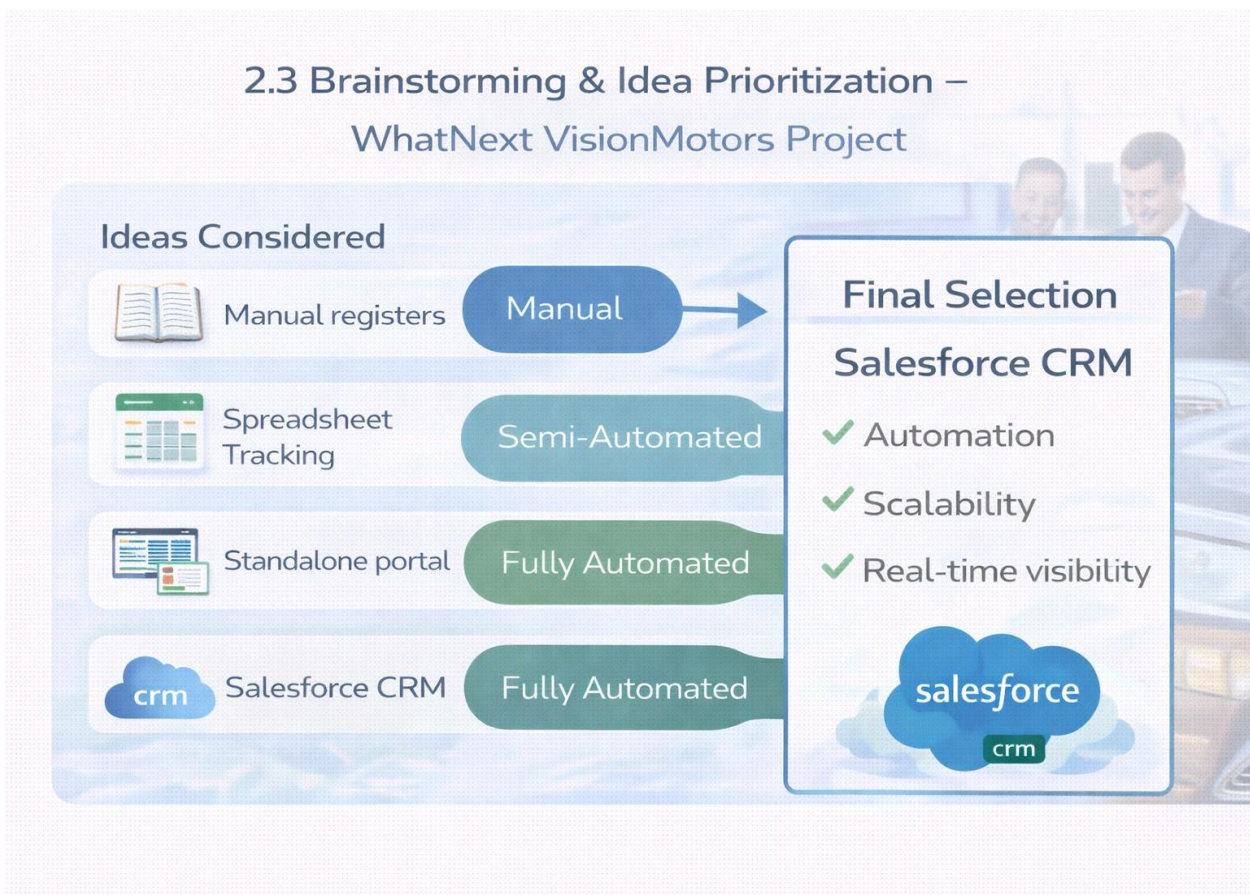
2.3 Brainstorming & Idea Prioritization

Ideas Considered:

- Manual registers, spreadsheet tracking, standalone web portal, Salesforce CRM.

Final Selection:

- Salesforce CRM was selected due to its built-in automation capabilities, highly scalable architecture, and real-time operational visibility.



3.REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages:

- Inquiry → Vehicle Selection → Order Placement → Dealer Assignment → Stock Validation → Test Drive → Confirmation → Reporting



3.2 Solution Requirements

Functional Requirements

FR No	Requirement
FR-1	Vehicle management
FR-2	Dealer management
FR-3	Customer order creation
FR-4	Stock validation
FR-5	Dealer auto-assignment
FR-6	Test drive scheduling
FR-7	Reports & dashboards

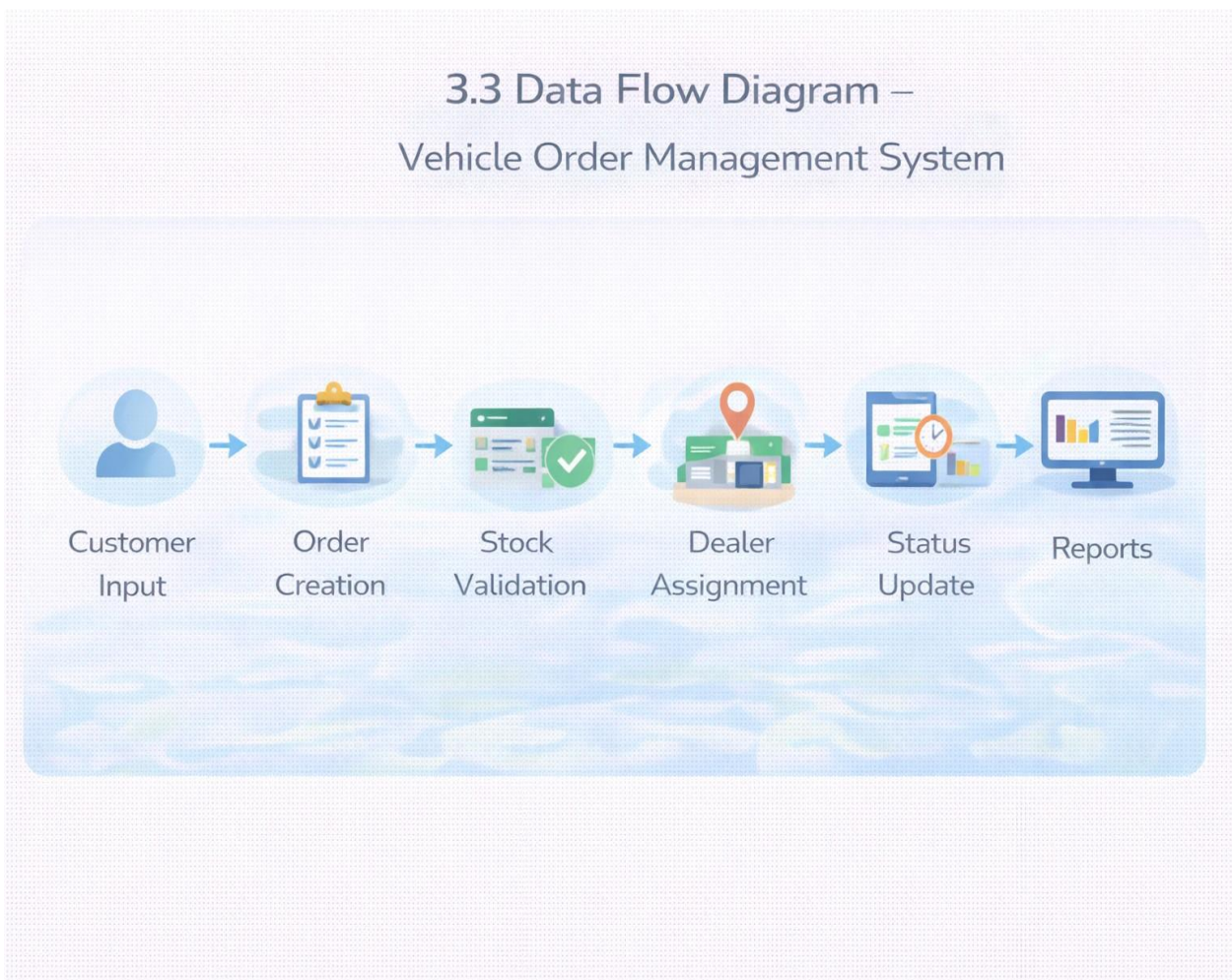
Non-Functional Requirements

NFR	Description
Usability	Lightning UI for intuitive and easy navigation
Security	Role-based access control to protect sensitive data.
Reliability	Accurate, error-free automated processes.
Performance	Fast order processing and query load times.
Availability	24/7 cloud access with minimal downtime
Scalability	Support for growing multi-dealer networks

3.3 Data Flow Diagram

Flow:

Customer Input → Order Creation → Stock Validation → Dealer Assignment → Status Update → Reports



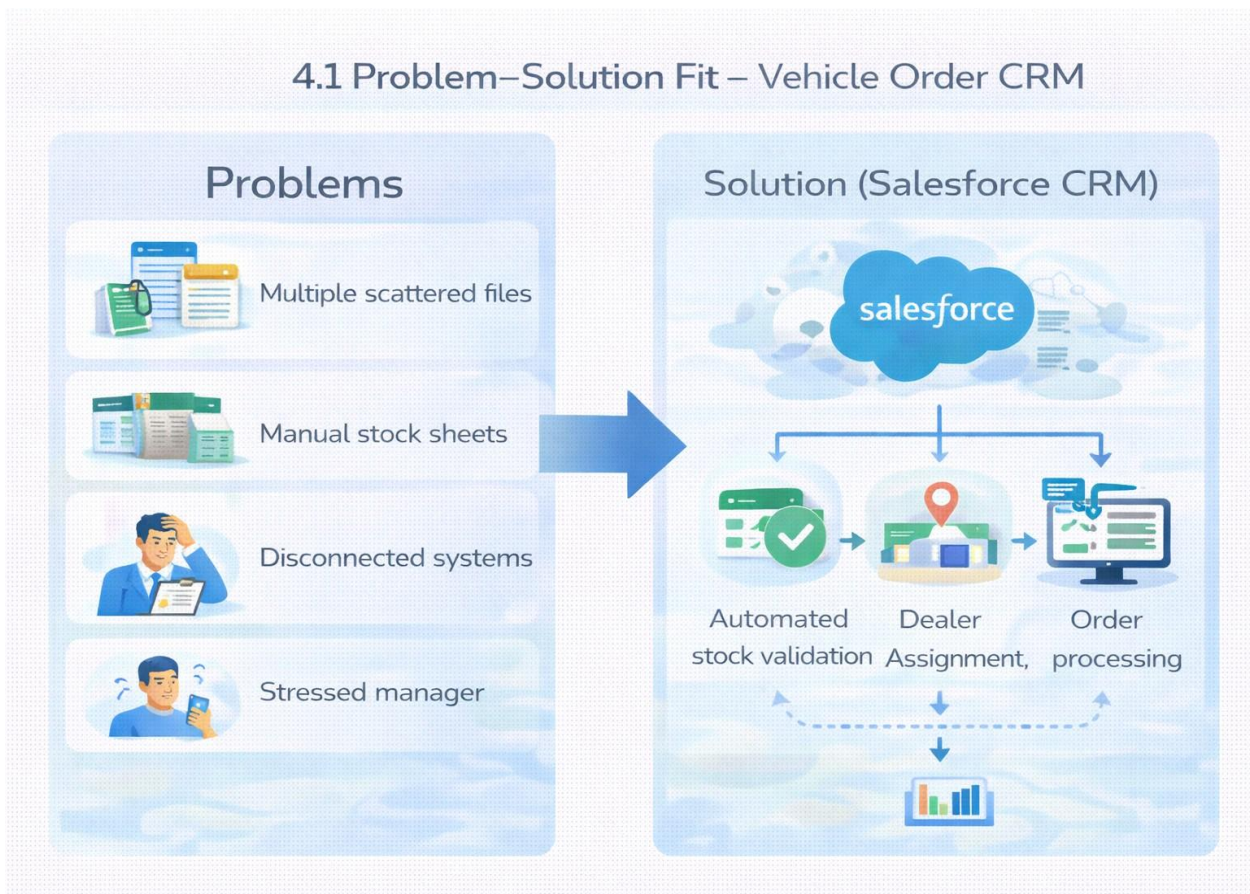
3.4 Technology Stack

Layer	Technology
UI	Salesforce Lightning
Logic	Salesforce Flows
Database	Salesforce Custom & Standard Objects
Automation	Validation Rules, Apex Triggers
Processing	Batch Apex, Scheduled Apex
Reporting	Salesforce Reports & Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The system eliminates fragmented operations by seamlessly integrating automated stock validation, intelligent dealer assignment, and streamlined order processing directly within the Salesforce ecosystem.



4.2 Proposed Solution

Functional Requirements

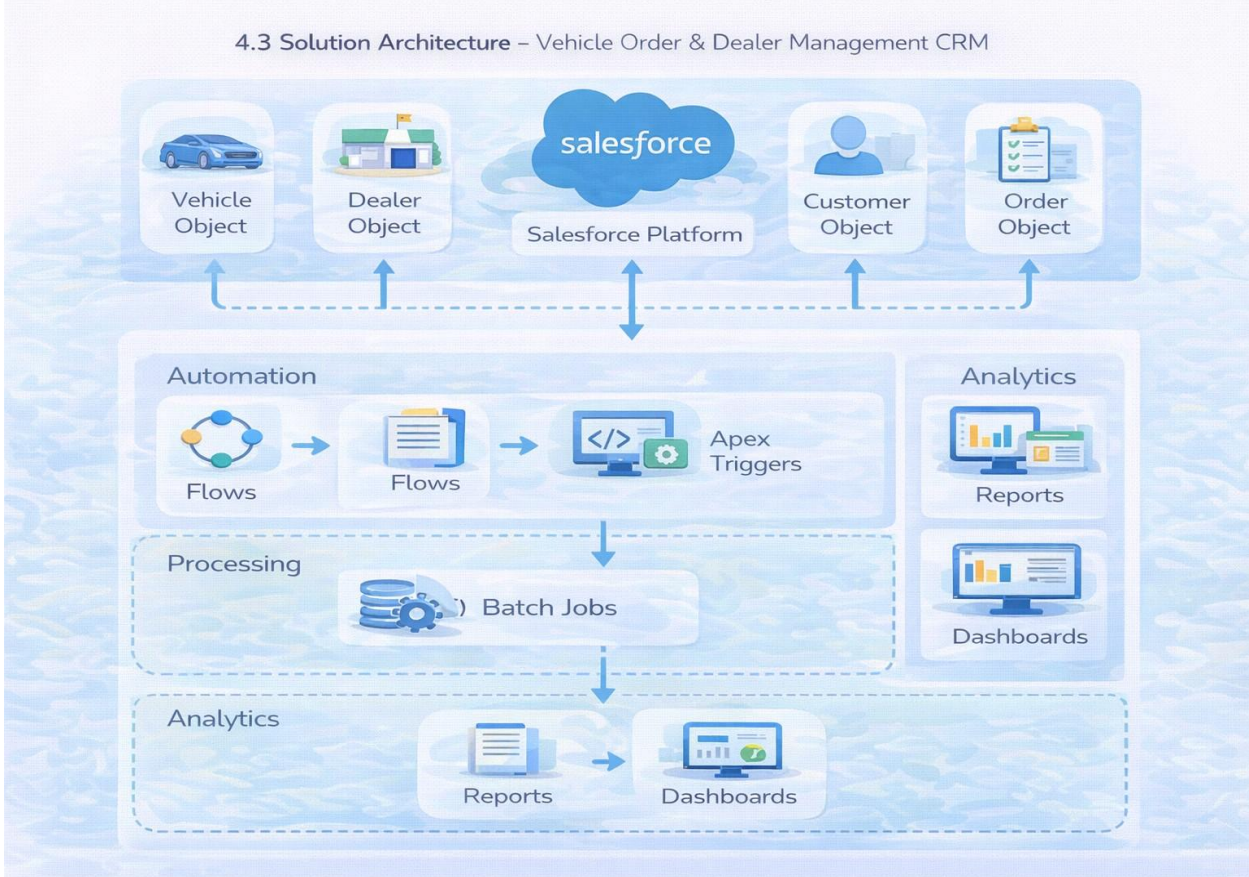
Parameter	Description
Problem	Manual, error-prone vehicle ordering processes.
Solution	Cloud-based Salesforce CRM integration.
Innovation	Automated geolocation-based dealer assignment.
Impact	Faster service delivery and error reduction.
Business Model	Dealer network distribution model.
Scalability	Multi-location and multi-region support.

4.3 Solution Architecture

Components:

- **Objects:** Vehicle, Dealer, Customer, Order, Test Drive, Service Request
- **Logic Components:** Flows, Apex Triggers, Batch Jobs
- **Analytics:** Reports, Dashboards

Sprint		Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1		Developer Setup	USN-1	As a system administrator, I want to create and configure a Salesforce developer account so that I can build and deploy the vehicle order management system.	3	High	Member
Sprint-2		Data Modeling	USN-2	As an admin, I want to create Vehicle, Dealer, Customer, Order,	5	High	Member



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning Methodology

- **Methodology:** Agile framework
- **Structure:** Epics → User Stories → Tasks
- **Execution:** Sprint-based cycles
- **Estimation:** Story Points and Velocity calculation

5.2 Product Backlog and Sprint Schedule

Sprint	Epic	USN	User Story / Task	Story Points	Priority	Assignee
Sprint-1	Developer Setup	USN-1	As a system admin, I want to configure a developer account to build the management system.	3	High	Prasanna Loganathan
Sprint-2	Data Modeling	USN-2	As an admin, I want to create Vehicle, Dealer, Customer, Order, and Service Request objects with proper relationships.	5	High	Kavin sharma
Sprint-2	Data Modeling	USN-3	As a staff member, I want dedicated tabs and a Lightning App for easy navigation across modules.	5	High	Absar jeelani
Sprint-3	Automation	USN-4	As a system, I want to validate stock availability and mandatory fields to prevent invalid orders.	3	High	Prasanna loganathan

Sprint	Epic	USN	User Story / Task	Story Points	Priority	Assignee
Sprint-3	Automation	USN-5	As a sales coordinator, I want flows to assign the nearest dealer and update order statuses automatically.	3	High	Kavin sharma
Sprint-4	Apex	USN-6	As a system, I want Apex triggers for instant order confirmation and dealer assignment.	5	High	Prasanna loganathan
Sprint-4	Apex	USN-7	As an inventory manager, I want batch jobs to update vehicle stock levels accurately.	5	High	Kavin sharma
Sprint-5	Reports	USN-8	As a manager, I want reports showing order status, dealer performance, and sales metrics.	4	High	Absar jeelani
Sprint-6	Dashboards	USN-9	As a manager, I want visual dashboards of sales metrics and performance to monitor operations.	4	Medium	Absar jeelani

6. Project Development Phase:

What you'll learn

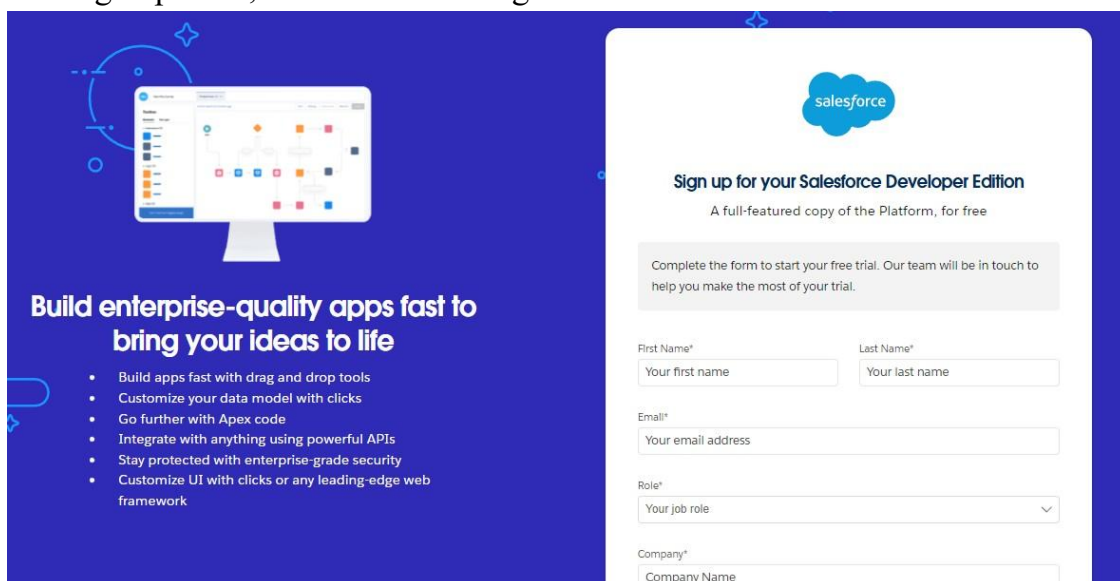
- **Data Modeling:** Designing robust relational database schemas.
- **Fields and Relationships:** Establishing Master-Detail and Lookup relationships.
- **Lightning App Builder:** Crafting customized, user-centric interfaces.
- **Record-Triggered Flows:** Implementing point-and-click process automation.
- **Apex and Apex Triggers:** Writing customized programmatic logic for complex validations.
- **Batch Apex:** Managing and updating large datasets efficiently.
- **Scheduled Apex:** Automating routine maintenance and communication tasks.

Milestone 1: Salesforce Account

Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to <https://developer.salesforce.com/signup>
2. On the sign up form, enter the following details :

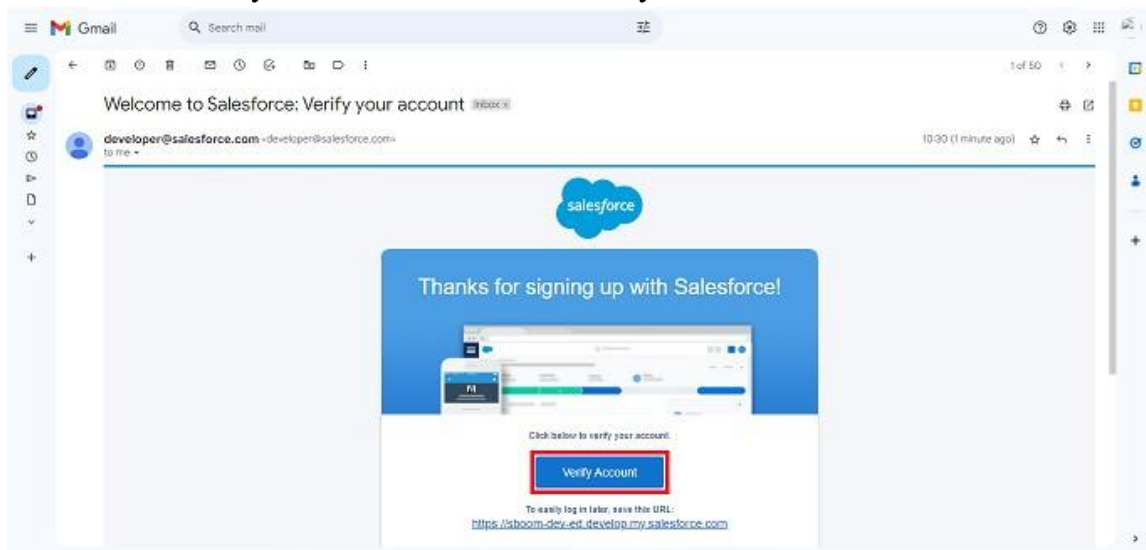


The image is a composite of two parts. On the left is a promotional graphic with a blue background. It features a computer monitor displaying a Salesforce interface with a flow diagram. Below the monitor, the text reads: "Build enterprise-quality apps fast to bring your ideas to life". Underneath this text is a bulleted list of features: "Build apps fast with drag and drop tools", "Customize your data model with clicks", "Go further with Apex code", "Integrate with anything using powerful APIs", "Stay protected with enterprise-grade security", and "Customize UI with clicks or any leading-edge web framework". On the right is a screenshot of the Salesforce Developer Edition sign-up form. The form has a white background with a blue Salesforce logo at the top. The heading is "Sign up for your Salesforce Developer Edition" with the subtext "A full-featured copy of the Platform, for free". Below this is a grey box with the text: "Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial." The form fields include: "First Name*" (text input), "Last Name*" (text input), "Email*" (text input), "Role*" (dropdown menu with "Your job role:" text), and "Company*" (text input with "Company Name" placeholder text).

- 1) First name & Last name
- 2) Email
- 3) Role: Developer
- 4) Company: College Name
- 5) County : India
- 6) Postal Code: pin code
- 7) Username: should be a combination of your name and company This need not be an actual email id, you can give anything in the format : [username@organization.com](#) Click on sign me up after filling these.

Activity 2: Account Activation

1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.

Change Your Password

Enter a new password for **lead@sb.oom**.
Make sure to include at least:

- ✓ 8 characters
- ✓ 1 letter
- ✓ 1 number

* New Password

Good

* Confirm New Password

Match

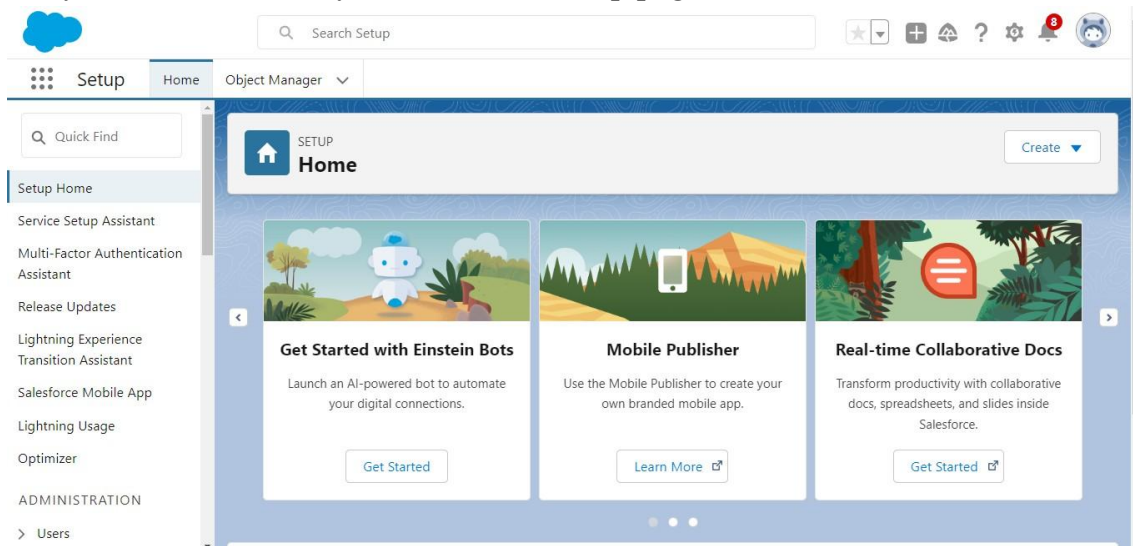
Security Question

▼ In what city were you born?

* Answer

Change Password

4. Then you will redirect to your salesforce setup page.



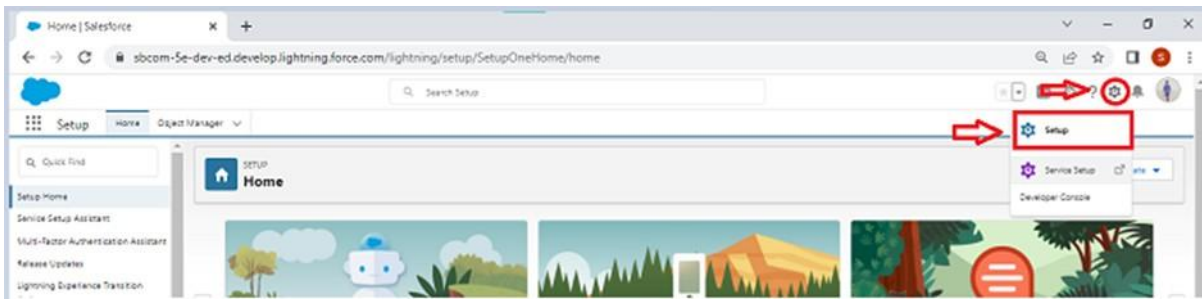
Milestone 2: Data Management-Objects

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:

Click on gear icon → click setup.



Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



- 1) Enter the label name → Vehicle
- 2) Plural label name → Vehicles
- 3) Enter Record Name Label and Format
 - Record Name → Vehicle Name
 - Data Type → Text

SETUP
New Custom Object

Custom Object Definition Edit Save Save & New Cancel

Custom Object Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account
 Plural Label Example: Accounts
☐ Starts with vowel sound

The Object Name is used when referencing the object via the API.

Object Name Example: Account
 Description

Context-Sensitive Help Setting ☒ Open the standard Salesforce.com Help & Training window
☐ Open a window using a Visualforce page

Content Name

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field

Record Name Example: Account Name
 Data Type **Warning:** If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

2. Click on Allow reports,
3. Allow search → **Save.**

Optional Features

☒ **Allow Reports** 
☐ Allow Activities
☐ Track Field History
☐ Allow in Chatter Groups
☐ Enable Licensing

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

☒ Allow Sharing
☒ Allow Bulk API Access
☒ Allow Streaming API Access

Deployment Status [What is this?](#)

☐ In Development
☒ Deployed

Search Status

When this setting is enabled, your users can find records of this object type when they search. [Learn more.](#)

☒ **Allow Search** 

Object Creation Options (Available only when custom object is first created)

☐ Add Notes and Attachments related list to default page layout
☐ Launch New Custom Tab Wizard after saving this custom object


Save Save & New Cancel


Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
 - 1) Enter the label name→ Vehicle Dealer
 - 2) Plural label name→ Vehicle Dealers
 - 3) Enter Record Name Label and Format
 - Record Name → Dealer Name
 - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

Activity 3: Creating remaining objects

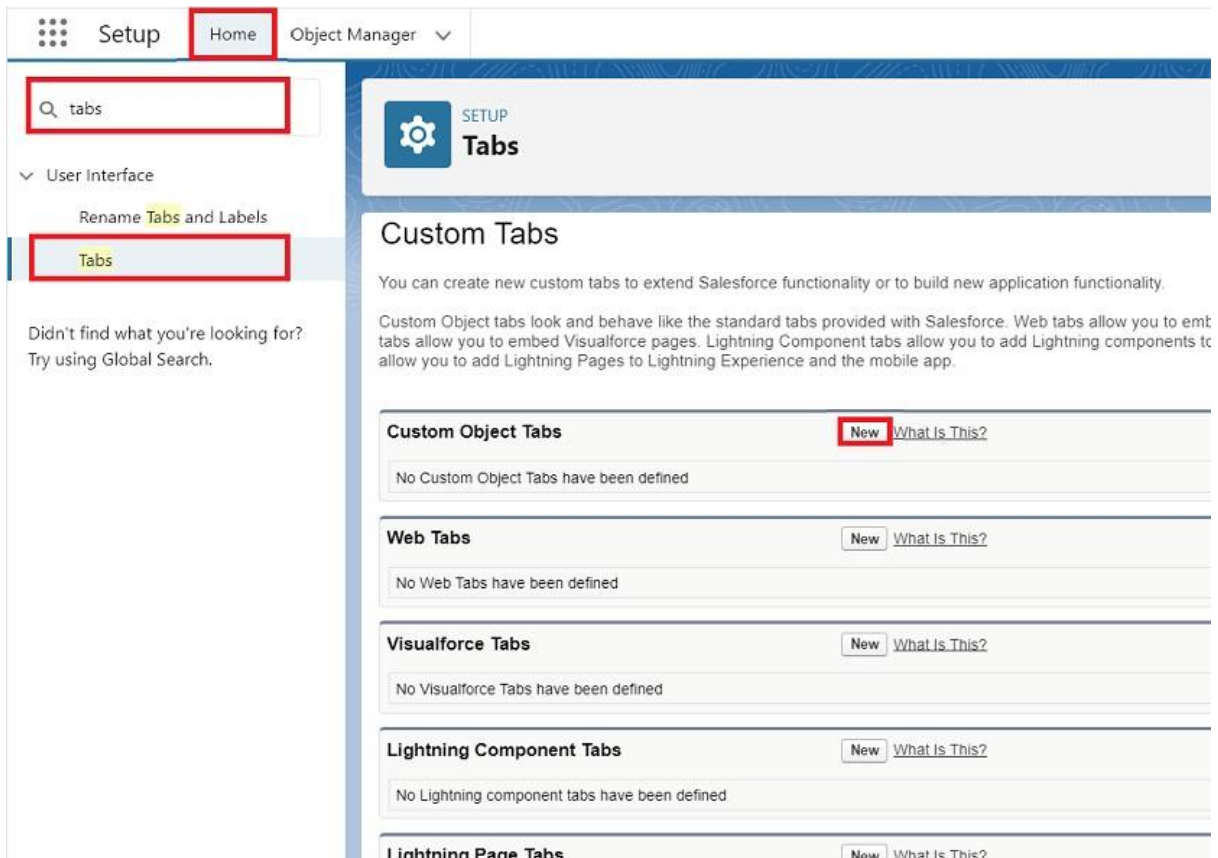
Creating remaining objects

Create remaining objects which have mentioned in the above table Data Management-Objects

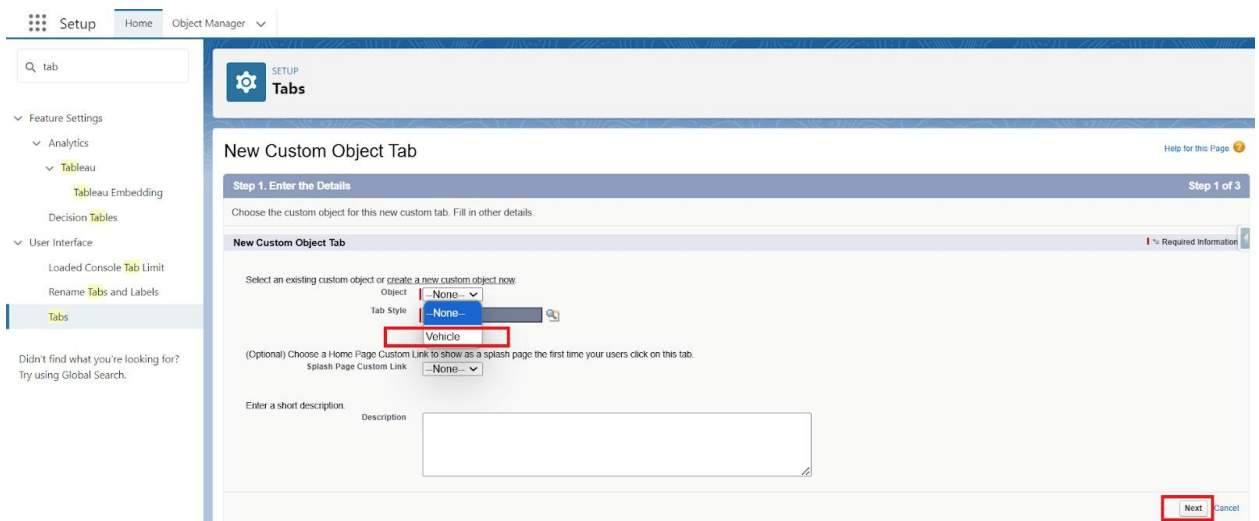
Milestone 3: Data Management-Tabs

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)



2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.



Activity 2:

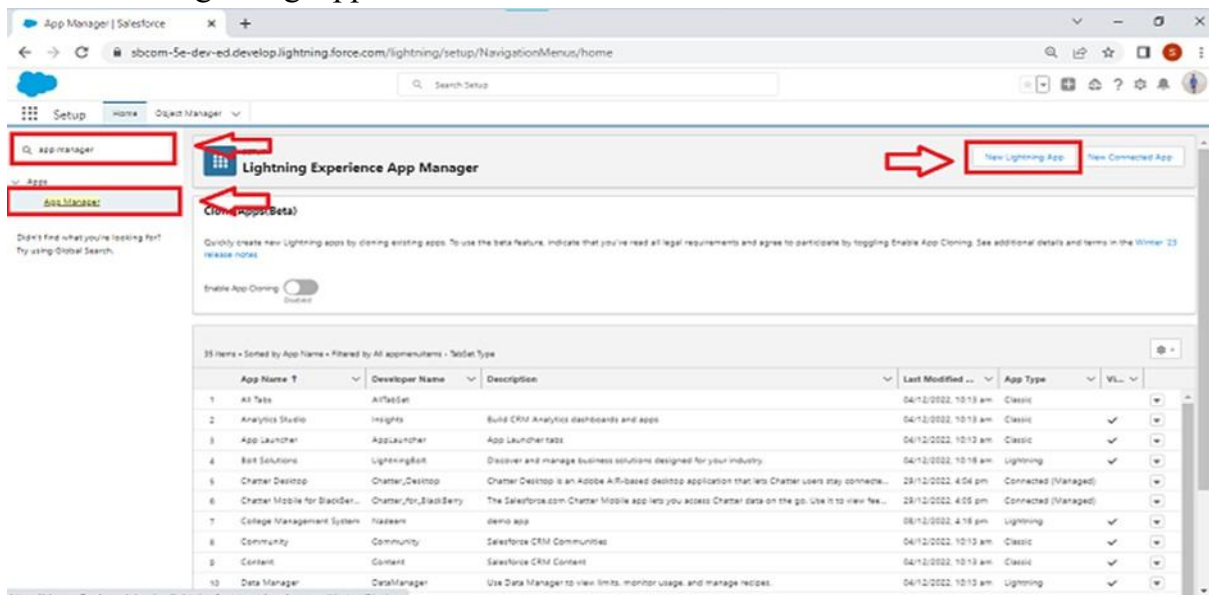
Now create tabs for Other objects as well which we have created above Objects.

Milestone 4: Data Management-App Manager

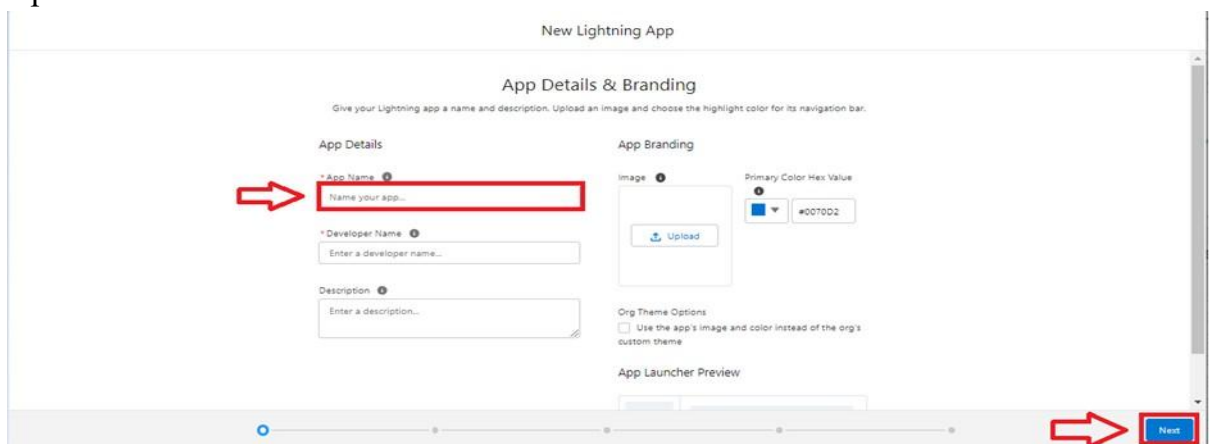
Activity 1: Create a Lightning App To

create a lightning app page:

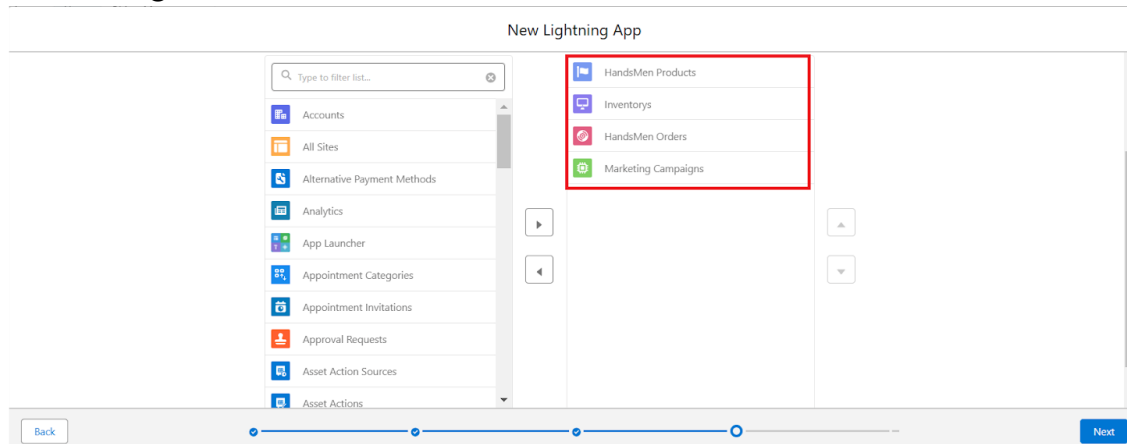
1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



2. Fill the app name in app details and branding as follow
App Name : WhatNext Vision Motors
Developer Name : this will auto populated
Description : Give a meaningful description
Image : optional (if you want to give any image you can otherwise not mandatory)
Primary color hex value : keep this default
3. Then click Next → (App option page) keep it as default → Next → (Utility Items)
keep it as default → Next.



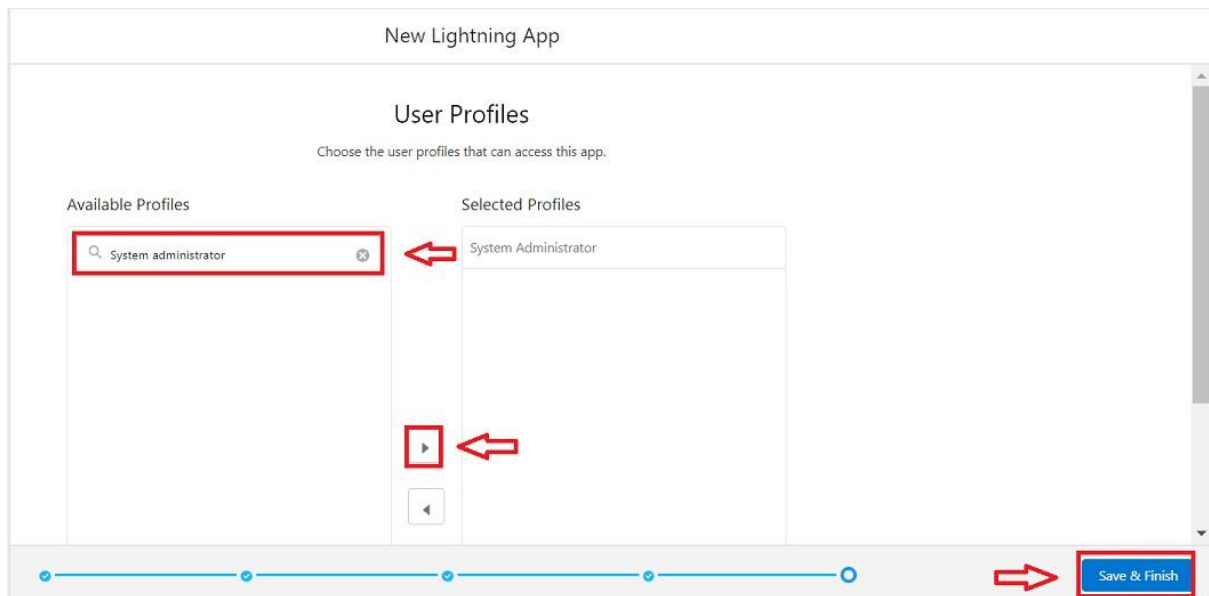
4. To Add Navigation Items:



Search the items in the search bar (Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

Note: select the custom object which we have created in the previous activity.

5. To Add User Profiles:



Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

Milestone 5: Data Management-Fields

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle_Name__c** (Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2. Vehicle_Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

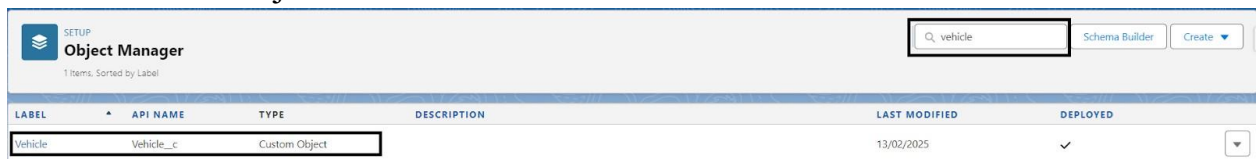
- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

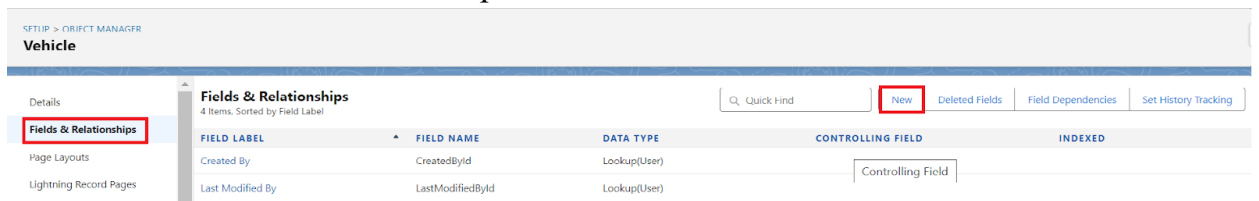
- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

Object Field Creation :

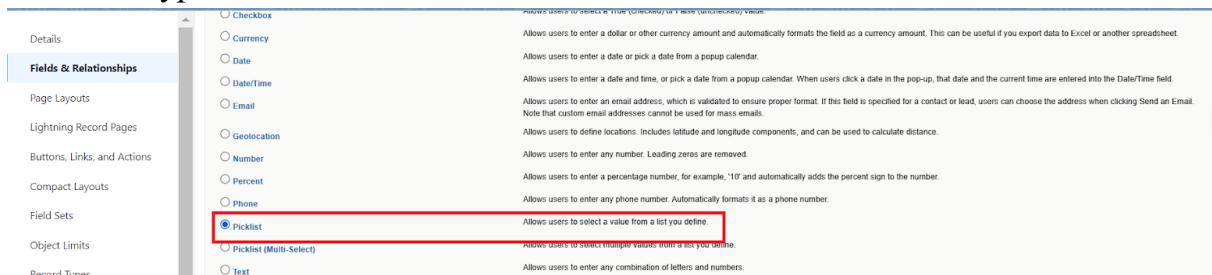
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”.



4. Click on Next

The screenshot shows the 'New Custom Field' configuration page for a 'Vehicle' object. The 'Fields & Relationships' sidebar is on the left. The main area is titled 'Step 2. Enter the details'. The 'Field Label' is 'Vehicle Model'. Under 'Values', the 'Enter values, with each value separated by a new line' option is selected, and a text box contains 'Sedan', 'SUV', 'RV', and 'etc'. The 'Field Name' is 'Vehicle_Model'. The 'Next' button is highlighted with a red box.

5. Fill the above as following:

- Field Label: Vehicle Modal
- Fill PickList Values
- Click on Next → Next → Save and new.

Object Field Creation-2 :

1. Repeat the steps 1 & 2 from the activity 1 2.

Select Data type as “Number”.

The screenshot shows the 'Number' data type selection screen. The 'Number' option is selected and highlighted with a red box. The right side of the screen provides detailed descriptions for various data types: Checkbox, Currency, Date, Date/Time, Email, Geolocation, Number (selected), Percent, and Phone.

3. Click on Next

The screenshot shows the 'Stock Quantity' field configuration screen. The 'Field Label' is 'Stock Quantity'. The 'Length' is set to '18' and 'Decimal Places' is set to '0'. The 'Field Name' is 'Stock_Quantity'. The 'Next' button is highlighted with a red box.

4. Fill the above as following:

- Field Label: Stock Quality
- Click on Next → Next → Save and new.

Relationship Field Creation :

Creating a Lookup Relationship in Vehicle Object to Dealer Object.

1. Select Data Type as Lookup Relationship.

Vehicle
New Custom Field

Step 1. Choose the field type

Specify the type of information that the custom field will contain.

Data Type

Select one of the data types below:

- ☐ None Selected
- ☐ Auto Number
- ☐ Formula
- ☐ Roll-Up Summary
- ☒ **Lookup Relationship**
- ☐ Master Detail Relationship

Creates a relationship that links this object to another object. The relationship field allows users to click on a lookup icon to select a value from a popup list. The other object is the source of the values in the list.

Creates a special type of parent-child relationship between this object (the child, or "detail") and another object (the parent, or "master") where:

- The relationship field is required on all detail records.
- The ownership and sharing of a detail record are determined by the master record.
- When a user deletes the master record, all detail records are deleted.

Next

2. Click Next

3. Select Dealer Object Form Related Filed.

Vehicle
New Relationship

Step 2. Choose the related object

Select the other object to which this object is related.

Related To: Dealer

Previous Next Cancel

Vehicle
New Relationship

Step 3. Enter the label and name for the lookup field

Field Label: Dealer

Field Name: Dealer

Description:

Help Text:

Previous Next Cancel

Vehicle
New Relationship

Step 4. Establish field-level security for reference field

Field Label	Dealer	Visible	Read-Only
Data Type	Lookup		
Field Name	Dealer		
Description			
Select the profiles to which you want to grant edit access to this field via field-level security. The field will be hidden from all profiles if you do not add it to field-level security.			
Field Level Security for Profile			
Analytics Cloud Integration User		☒	☐
Analytics Cloud Security User		☒	☐
Contract Manager		☒	☐
Cross Org Data Proxy User		☒	☐
Custom Marketing Profile		☒	☐

Previous Next Cancel

4. Click Next and Save.

More Fields Creation

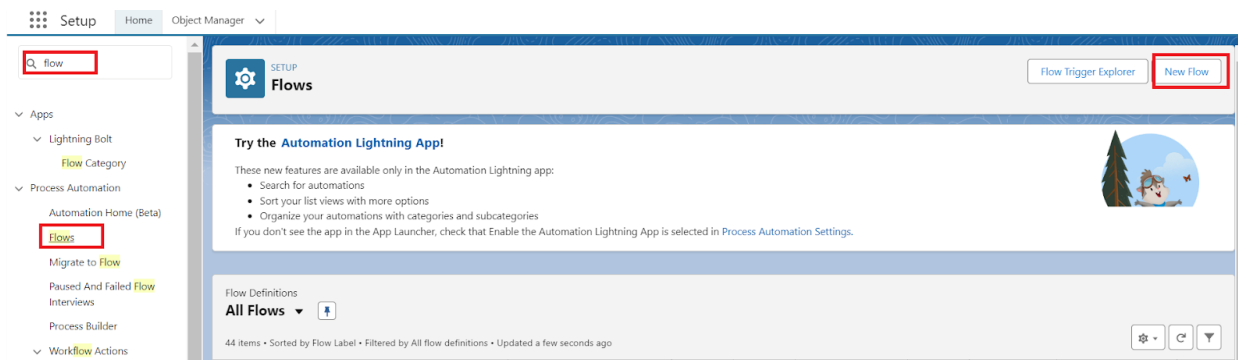
Create remaining fields as mentioned in the Data Management-Fields

Milestone 6: Automation

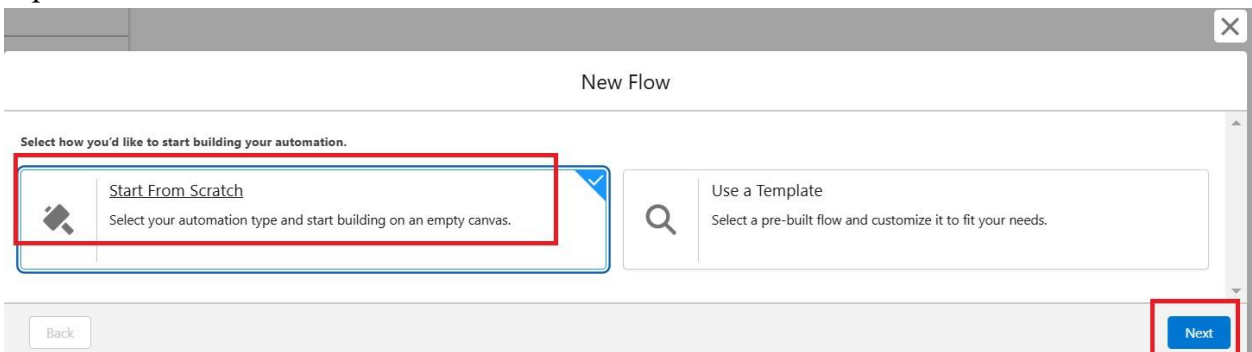
Flow Creation :

Creating a record triggered flow to assign nearest dealer to the customer's location

Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Start From Scratch and click Next.



Step 3 : Select Record-Triggered Flow and click Create

Select Type

Recommended



Screen Flow
Guides users through a business process that's launched from Lightning pages, Experience Cloud sites, quick actions, and more.



Record-Triggered Flow
Launches when a record is created, updated, or deleted. This autolaunched flow runs in the background.



Schedule-Triggered Flow
Launches at a specified time and frequency for each record in a batch. This autolaunched flow runs in the background.



Platform Event—Triggered Flow
Launches when a platform event message is received. This autolaunched flow runs in the background.



Autolaunched Flow (No Trigger)
Launches when invoked by Apex, processes, REST API, and more. This autolaunched flow runs in the background.



Record-Triggered Orchestration
Launches when a record is created or updated. An orchestration lets you create a multi-step, multi-user process.

All Flow Types

Autolaunched Flow (No Trigger)

Autolaunched Orchestration (No Trigger)

[Back](#) [Create](#)

Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :

All Conditions Are Met (AND)

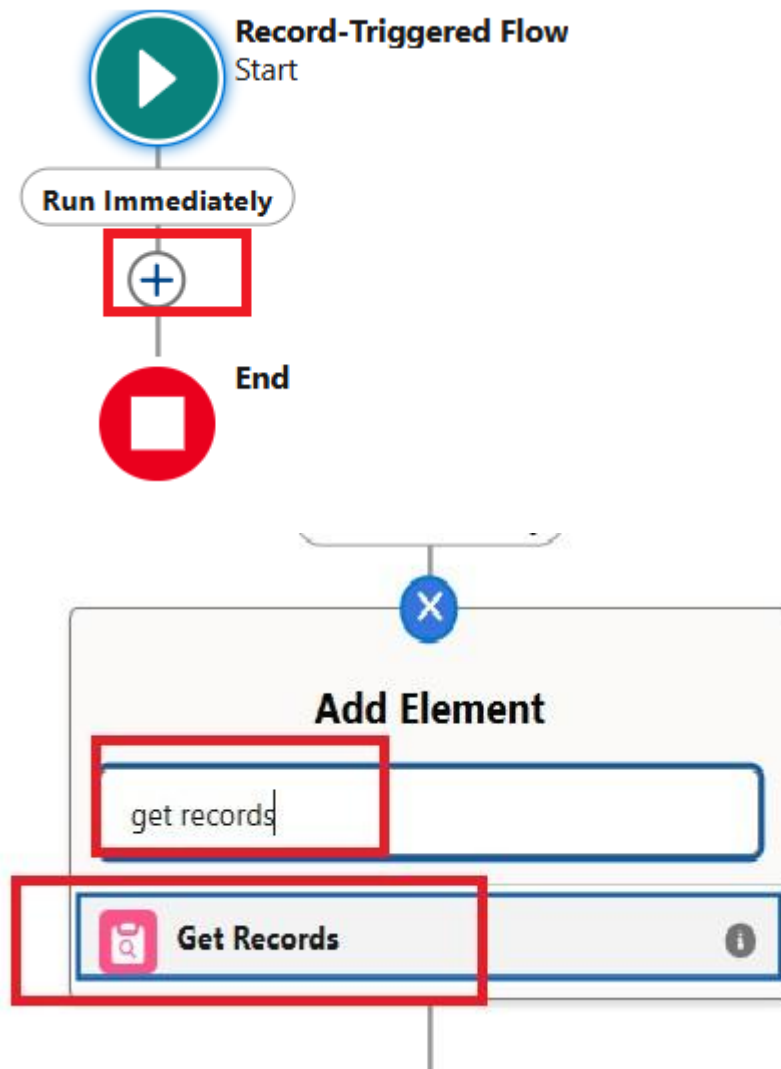
Filed : Status__c

Operator : Equals

Value : Pending

```
app = Flask(__name__)
```

Step 5 : Click + → Select **Get Records**



Step 6 : **Label** : Get Customer Information

- Object : Vehicle Customer ● Condition :
- Field : Id
- Operator : Equals
- Value : {!\$Record.Vehicle_Customer__c}

 Get Records

* Label

Get Customer Information.

* API Name

Get_Customer_Information

Description

Get Records of This Object

* Object

Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements

All Conditions Are Met (AND)

Field	Operator	Value
Id	Equals	Aa \$Record > Vehicle Customer X

+ Add Condition

Sort Vehicle Customer Records

Step 7 : Click + → Select **Get Records**

- **Label** : Get Nearest Dealer ● **Object** : Vehicle Dealer ● **Condition** :
- **Field** : Dealer_Location__c
- **Operator** : Equals
- **Value** : {!Get_Customer_Information.Address__c}



Get Records

*Label

Get Nearest Dealer

*API Name

Get_Nearest_Dealer

Description

Get Records of This Object

*Object

Vehicle Dealer

Filter Vehicle Dealer Records

Condition Requirements

All Conditions Are Met (AND) ▼

Field

Dealer_Location__c

Operator

Equals ▼

Value

Vehicle Customer from Get_Cu... ✕

+ Add Condition

Step 8 : Click + → Select Update Records

- **Label** : Assign Dealer to Order
- *How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection
- Select Record(s) to Update : {!Get_Nearest_Dealer}

 **Update Records** ➤

*** Label** *** API Name** ⓘ

Assign Dealer to Order Assign_Dealer_to_Order

Description

*** How to Find Records to Update and Set Their Values**

☐ Use the vehicle order record that triggered the flow
☐ Update records related to the vehicle order record that triggered the flow
☒ Use the IDs and all field values from a record or record collection
☐ Specify conditions to identify records, and set fields individually

Select Record(s) to Update

Record or Record Collection

 Vehicle Dealer from Get Nearest Dealer ✕

Make sure that each record has an ID. Otherwise the flow can't find the records to update, and it fails. ⓘ

Step 9 : Click Save and Give label Name and Activate Flow.

- Label Name : Auto Assign Dealer

Save as

Save As

A New Version

⬆ ⬇ ⬆

*** Flow Label** *** Flow API Name** ⓘ

Auto Assign Dealer Auto_Assign_Dealer

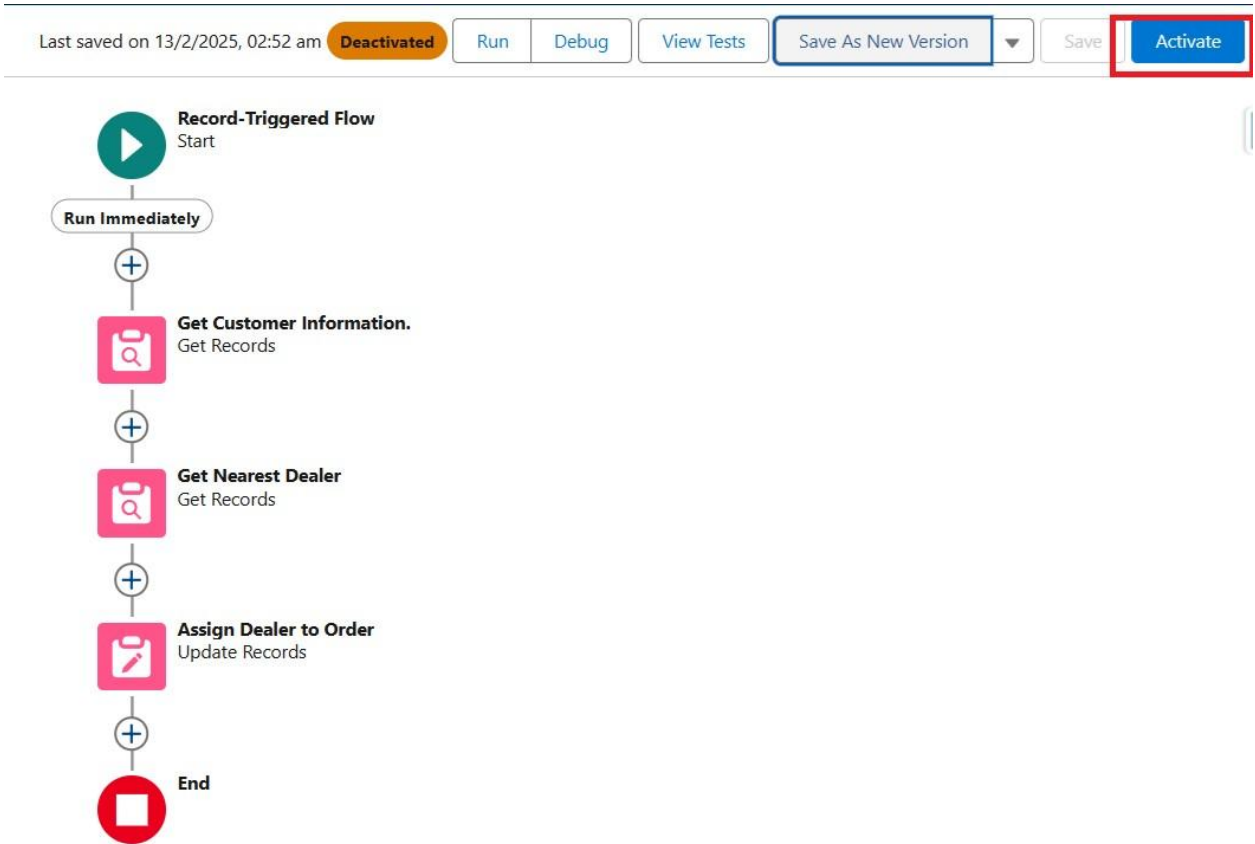
Description

[Show Advanced](#)

Save

Cancel

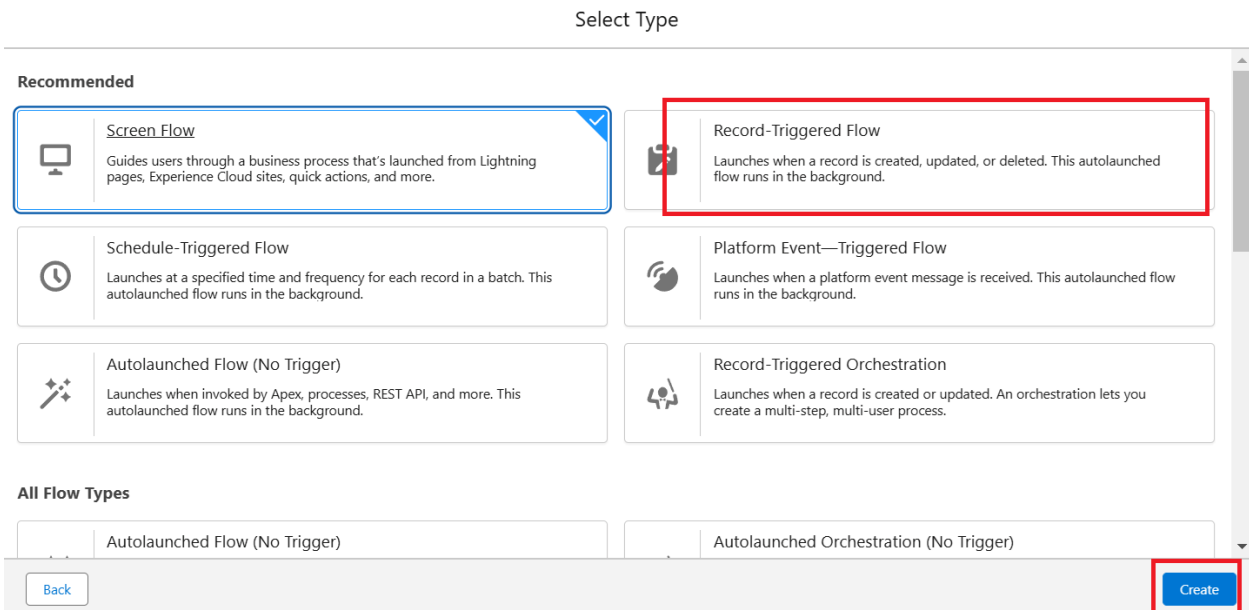
Step 10 : Activate Flow



Record-Trigger Flow Creation :

Creating a record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow and click Create



Step 2 : Select Vehicle Test Drive Object

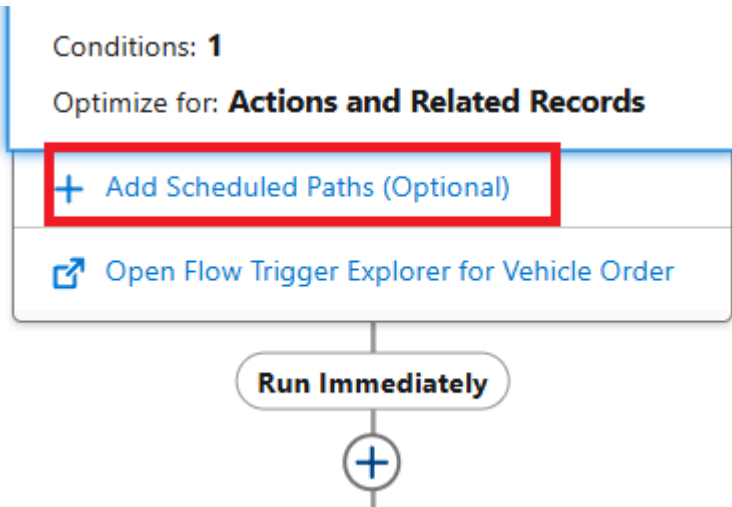
Trigger the Flow When : A record is created or updated

Set Entry Condition :

- All Conditions Are Met (AND)
- Filed : Status__c
- Operator : Equals
- Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

- Label: Reminder Before Test Drive. ● Time Source: Test_Drive_Date__c ● Offset Number: 1.
- Offset Options: Days Before.
- Click Done.



Configure Scheduled Paths

SCHEDULED PATHS

+ Add a scheduled path if you want a flow path to run a certain amount of time after the triggering record is created or updated. That time can be based on the triggering event or on a specified date or date/time field in the record. A separate flow interview runs for each scheduled path. Each scheduled path interview is queued to run at the scheduled time, but can be delayed until system resources become available. Path labels appear on connectors in the canvas.

Run Immediately

Reminder Before Test Drive.

SCHEDULED PATH DETAILS

Delete Path

* Path Label

Reminder Before Test Drive.

* API Name

Reminder_Before_Test_Drive

* Time Source

Vehicle_Test_Drive__c: Test Drive Date

* Offset Number

1

* Offset Options

Days Before

> Show advanced options

Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!\$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

Get Records

* Label: Get Customer Information

* API Name: Get_Customer_Information

Description:

Get Records of This Object

Object: Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements: All Conditions Are Met (AND)

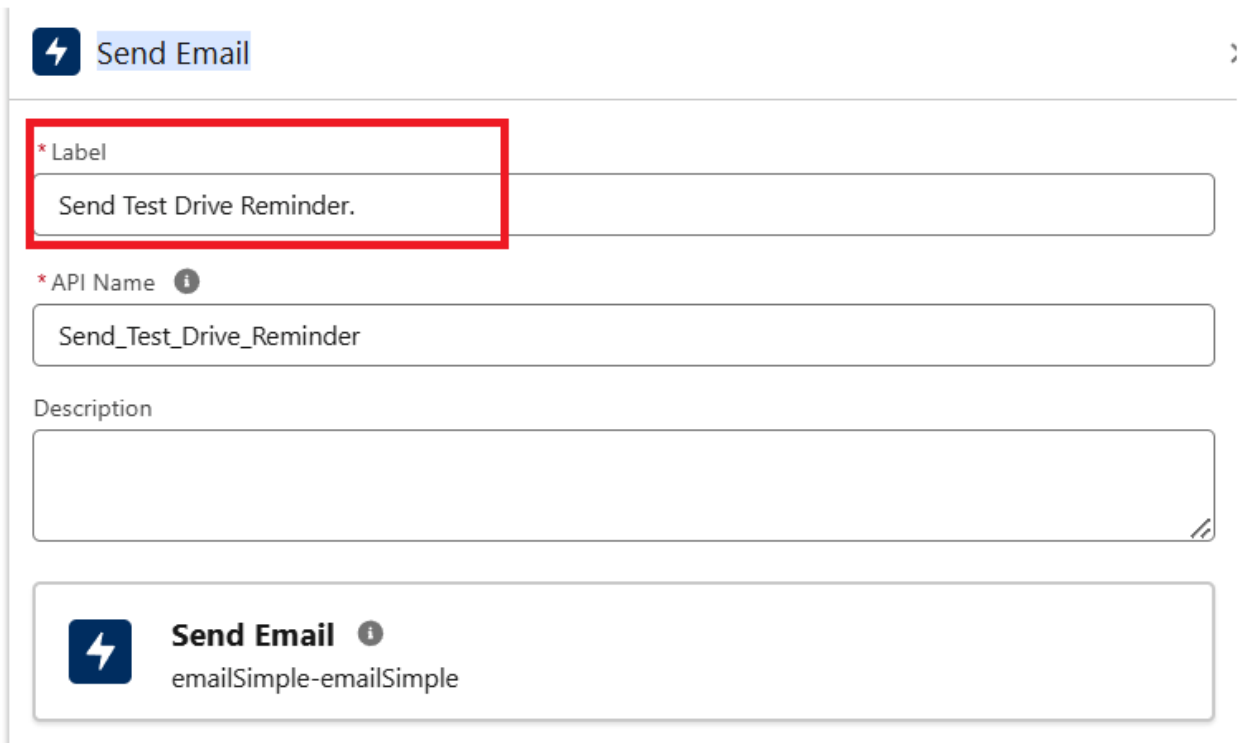
Field	Operator	Value
Id	Equals	{!\$Record > Vehicle Customer}

+ Add Condition

Sort Vehicle Customer Records

Step 5: Send Reminder Email

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email_c}
6. Rich-Text-Formatted Body : True



Send Email

* Label

Send Test Drive Reminder.

* API Name ⓘ

Send_Test_Drive_Reminder

Description

Send Email ⓘ
emailSimple-emailSimple

7. Body : Create Variable
- 8 . Api Name : EmailSent

Send Email

A_a Body 	 EmailSent 	 Included
A_a CC Recipient Address List		 Not Included
A_a Email Template ID		 Not Included
 Log Email on Send		 Not Included
A_a Recipient Address Collection		 Not Included

Step 6 : Click save

- **Label Name** : Test Drive Reminder

Save as

Save As

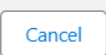
A New Version 

Flow Label	* Flow API Name 
Test Drive Reminder	Test_Drive_Reminder

Description



[Show Advanced](#)

Step 7 : Activate Flow

Run

Debug

View Tests

Save As New Version

▼

Save

Activate



Flow Page After Creation :

□

Select Elements

↶

↷

🗑

⚙

Auto-Layout

▼

Last saved on 13/2/2025, 03:57 am

Active

Run

Debug

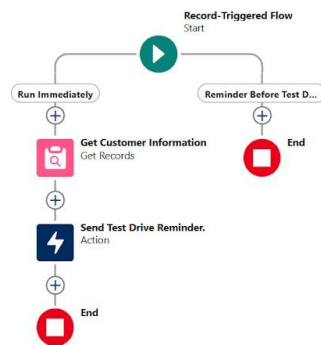
View Tests

Save As New Version

▼

Save

Deactivate



−

⌕

+

Test your Flow :

Inbox

76,244

Starred

Snoozed

Important

Sent

Drafts

Categories

Social

3,451

Updates

83,750

Forums

Be careful with this message.

This may be a spoofed message. The message claims to have been sent from your account, but Gmail couldn't verify the actual source. Avoid clicking links or replying with sensitive information, unless you are sure that you actually sent this message. (No need to reset your password, the real sender does not actually have access to your account.)

Report spam

Looks safe

Dear test KHIT 123434,

This is a reminder that your test drive for a08dM00000D0ck9QAB is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email]. Thank you, WhatNext Vision Motors Team"

Send Email

A_a Recipient Address List ⓘ

A_a ...er from Get Customer Information > Email ✕

Included

A_a Recipient ID

Vehicle Customer from Get Customer Information > Email

Not Included

A_a Related Record ID

Not Included

🔓 Rich-Text-Formatted Body ⓘ

True ✕

Included

A_a Sender Email Address

Not Included

A_a Sender Type

Not Included

A_a Subject ⓘ

Reminder: Your Test Drive is Tomorrow! 🔍

Included

🔓 Use Line Breaks ⓘ

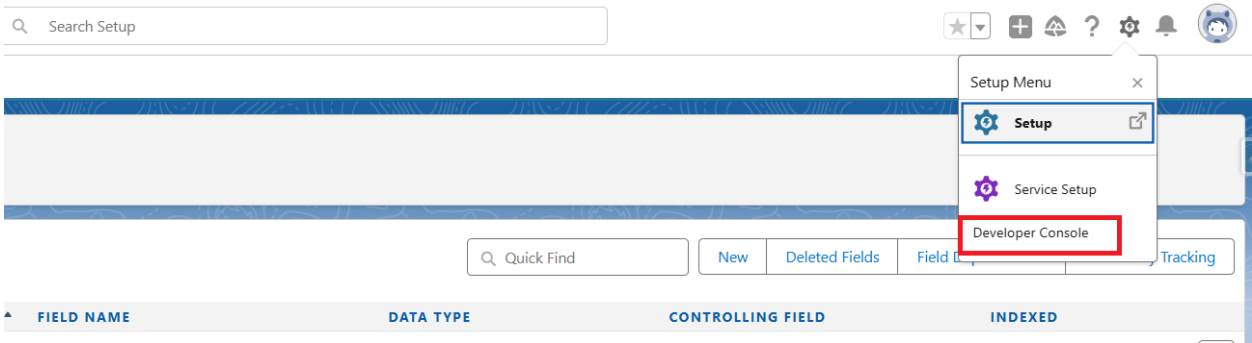
True ✕

Included

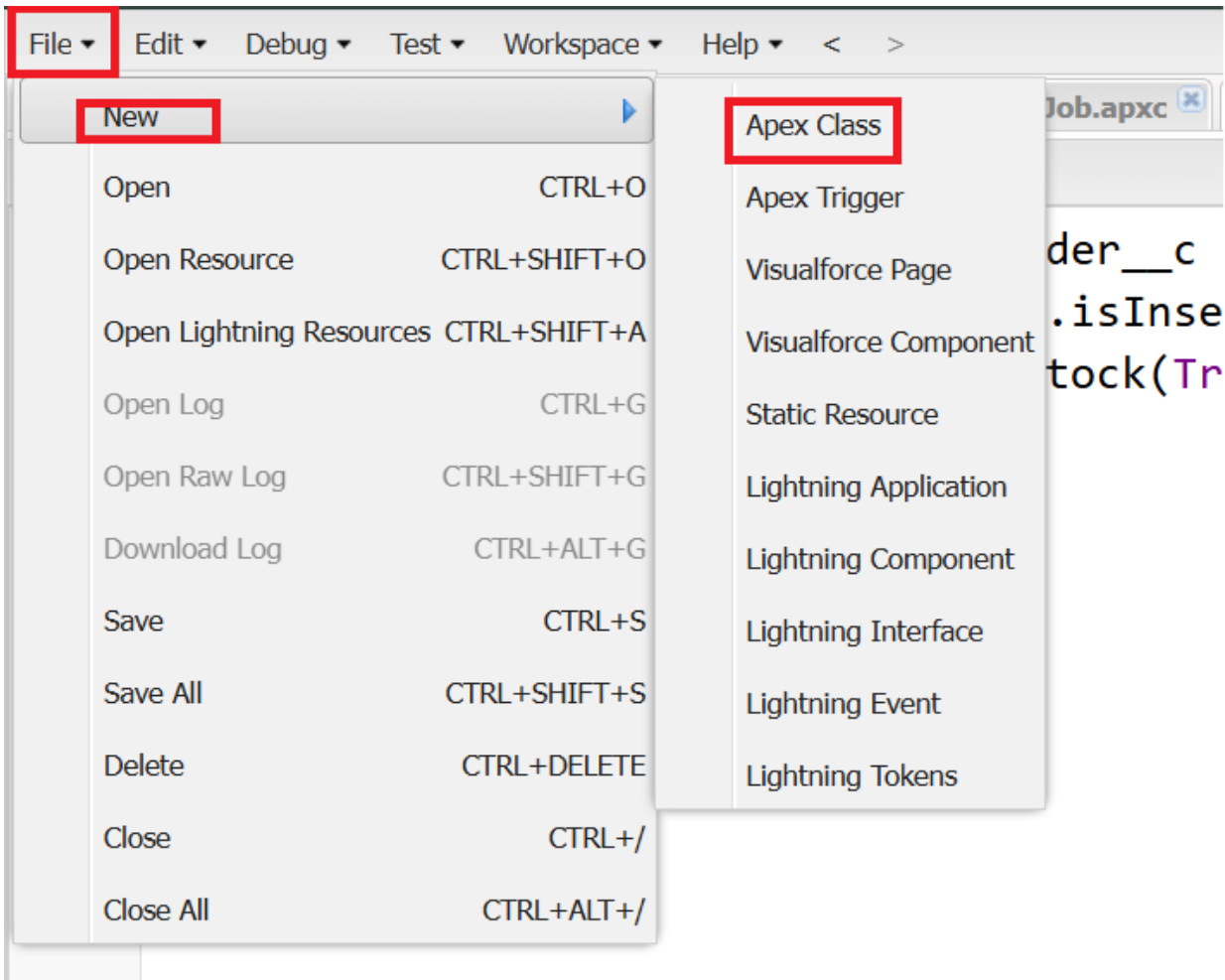
Milestone 7: Apex and Batch Class

Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon

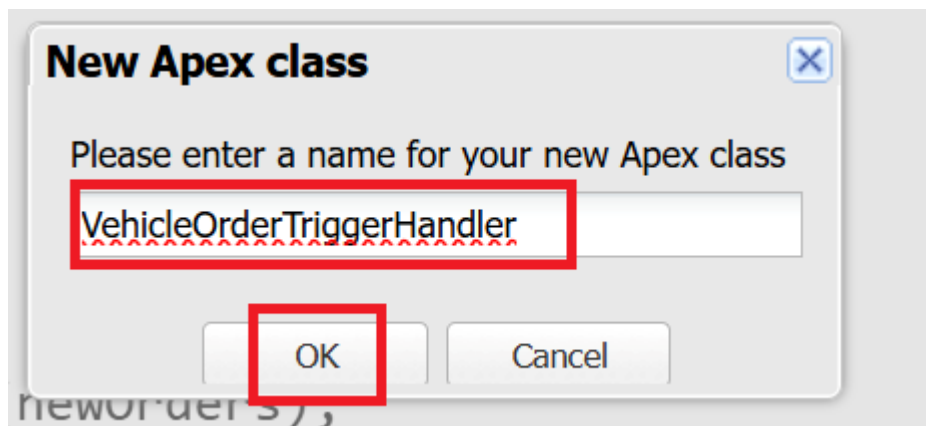


Step 2 : click File then Select New then Select Apex.



Step 3 : Give Name For Apex Class

- Class Name : VehicleOrderTriggerHandler



Step 4 : Write Apex Code

```

File • Edit • Debug • Test • Workspace • Help • < >
OrderTriggerHandler.aspx • OrderTrigger.aspx • InventoryBatchJob.aspx • VehicleOrderTriggerHandler.aspx • VehicleOrderTrigger.aspx
Code Coverage: None • API Version: 52
Go To

1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(List<Vehicle_Order_c> newOrders, Map<Id, Vehicle_Order_c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
4         if (isBefore) {
5             if (isInsert || isUpdate) {
6                 preventOrderIfOutOfStock(newOrders);
7             }
8         }
9
10        if (isAfter) {
11            if (isInsert || isUpdate) {
12                updateStockOnOrderPlacement(newOrders);
13            }
14        }
15    }
16
17    // Method to prevent orders when the vehicle is out of stock
18    private static void preventOrderIfOutOfStock(List<Vehicle_Order_c> orders) {
19        Set<Id> vehicleIds = new Set<Id>();
20        for (Vehicle_Order_c order : orders) {
21            if (order.Vehicle_c != null) {
22                vehicleIds.add(order.Vehicle_c);
23            }
24        }
25
26        if (!vehicleIds.isEmpty()) {
27            Map<Id, Vehicle_c> vehicleStockMap = new Map<Id, Vehicle_c>();
28            for (Vehicle_c vehicle : [SELECT Id, Stock_Quantity_c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
29                vehicleStockMap.put(vehicle.Id, vehicle);
30            }
31
32            for (Vehicle_Order_c order : orders) {
33                if (vehicleStockMap.containsKey(order.Vehicle_c)) {
34                    Vehicle_c vehicle = vehicleStockMap.get(order.Vehicle_c);
35                    if (vehicle.Stock_Quantity_c <= 0) {
36                        order.addError('This vehicle is out of stock. Order cannot be placed.');

Logs Tests Checkpoints Query Editor View State Progress Problems



| User                                                          | Application | Operation | Time | Status | Read | Size |
|---------------------------------------------------------------|-------------|-----------|------|--------|------|------|
| <input type="text"/> Filter Click here to filter the log list |             |           |      |        |      |      |


```

Source Code :

```
public class VehicleOrderTriggerHandler {
```

```

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert,
Boolean isUpdate) {

        if (isBefore) {            if (isInsert || isUpdate)

{

preventOrderIfOutOfStock(newOrders);

        }

    }

        if (isAfter) {            if (isInsert || isUpdate) {

updateStockOnOrderPlacement(newOrders);

        }

    }

}

// Method to prevent orders when the vehicle is out of stock    private static void
preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {        Set<Id>
vehicleIds = new Set<Id>();        for (Vehicle_Order__c order : orders) {            if
(order.Vehicle__c != null) {                vehicleIds.add(order.Vehicle__c);

            }

        }

        if (!vehicleIds.isEmpty()) {

            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();

            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {                vehicleStockMap.put(vehicle.Id, vehicle);

```

```

    }

    for (Vehicle_Order__c order : orders) {
        if
(vehicleStockMap.containsKey(order.Vehicle__c)) {

            Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
            if (vehicle.Stock_Quantity__c <= 0) {
                order.addError('This vehicle is
out of stock. Order cannot be placed.');
```

```

            }
        }
    }
}

// Method to update vehicle stock when an order is placed private static void
updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id>
vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if
(order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();

        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }
    }
}

```

```

        List<Vehicle__c> vehiclesToUpdate = new
List<Vehicle__c>();        for (Vehicle_Order__c order : orders) {
if (vehicleStockMap.containsKey(order.Vehicle__c)) {

        Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
if (vehicle.Stock_Quantity__c > 0) {
vehicle.Stock_Quantity__c -= 1;
vehiclesToUpdate.add(vehicle);

        }

    }

    }

    if (!vehiclesToUpdate.isEmpty()) {
update vehiclesToUpdate;

    }

    }

    }
}

```

Vehicle Order
VC-0003

* = Required Information

Vehicle Order Name
VC-0003

Owner
Abdul Trainer

Vehicle Customer
test KHIT 123434

Vehicle
1erwe

Order Date
21/02/2025

Status
Pending

Created By
Abdul Trainer, 13/02/2025, 5:34 am

By
Abdul Trainer, 13/02/2025, 5:41 am

We hit a snag.

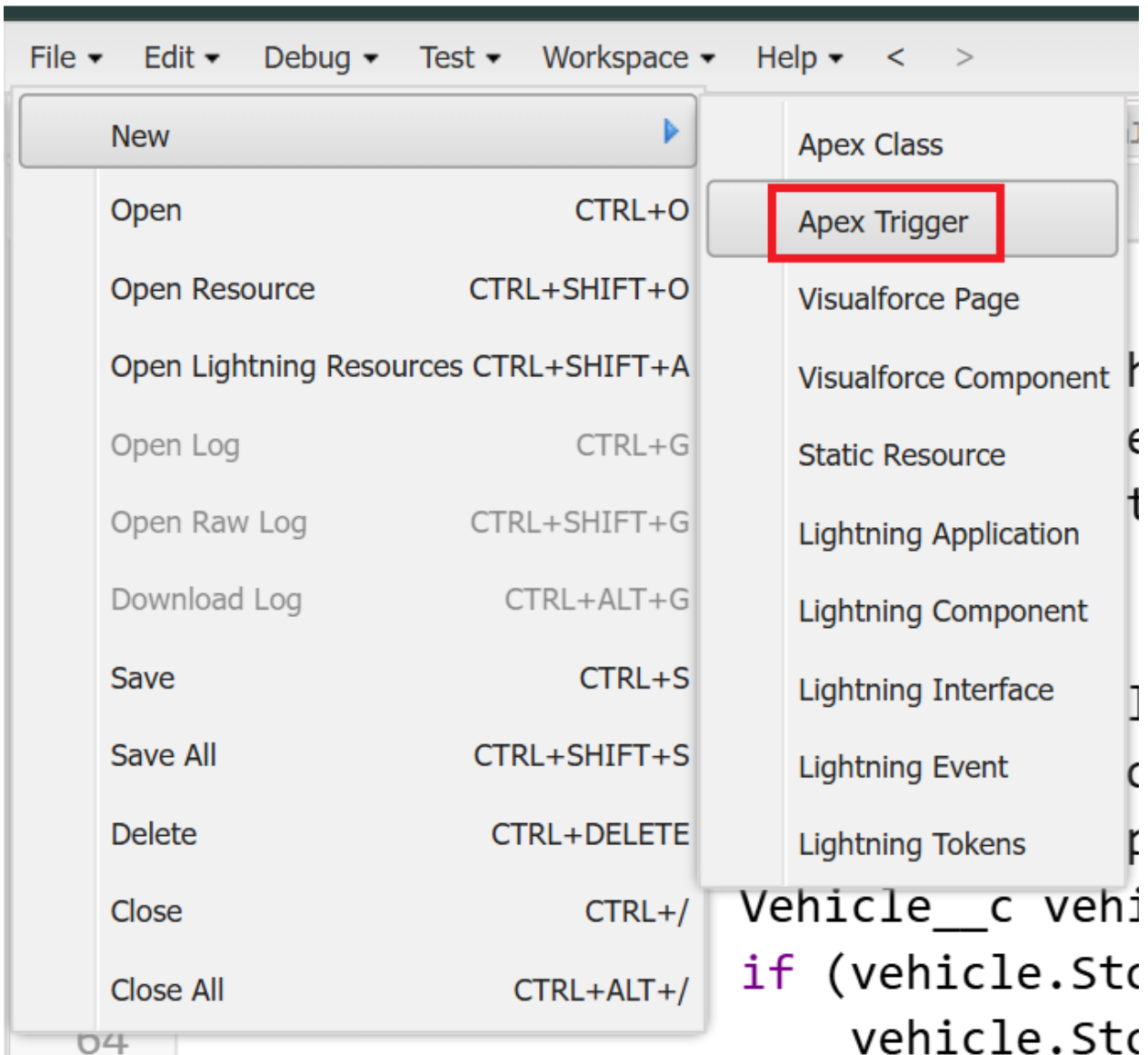
Review the errors on this page.

- This vehicle is out of stock. Order cannot be placed.

Cancel

Save

Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object

New Apex Trigger

Name: VehicleOrderTrigger

sObject: Vehicle_Order__c

Submit

Step 7 : Call Apex Class in Trigger Class

Source Code:

trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {

VehicleOrderTriggerHandler.handleTrigger([Trigger.new](#), Trigger.oldMap, Trigger.isBefore, Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate); }

```
Code Coverage: None | API Version: 62
1 trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
2   VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore, Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
3 }
4
```

Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.

New Apex class



Please enter a name for your new Apex class

OK

Cancel

Code Coverage: None API Version: 62

```
1 global class VehicleOrderBatch implements Database.Batchable<SObject> {
2
3     global Database.QueryLocator start(Database.BatchableContext bc) {
4         return Database.getQueryLocator([
5             SELECT Id, Status__c, Vehicle__c
6             FROM Vehicle_Order__c
7             WHERE Status__c = 'Pending'
8         ]);
9     }
10
11     global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
12         Set<Id> vehicleIds = new Set<Id>();
13
14         for (Vehicle_Order__c order : orderList) {
15             if (order.Vehicle__c != null) {
16                 vehicleIds.add(order.Vehicle__c);
17             }
18         }
19
20         if (!vehicleIds.isEmpty()) {
21             Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
22             for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
23                 vehicleStockMap.put(vehicle.Id, vehicle);
24             }
25
26             List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
27             List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
28
29             for (Vehicle_Order__c order : orderList) {
30                 if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

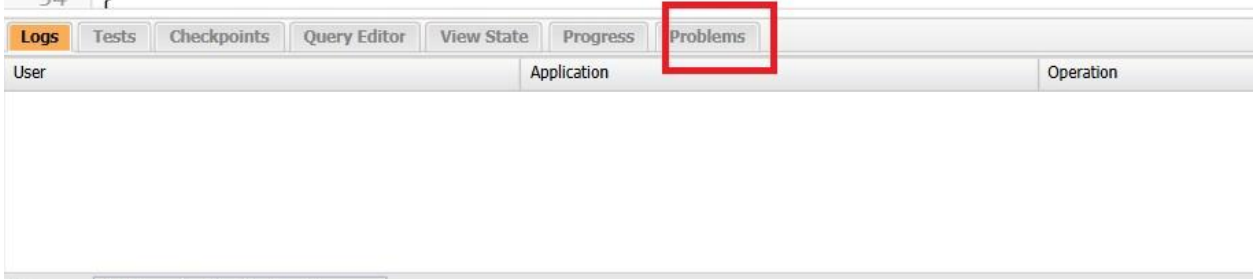
```

30  if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31      Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
32  if (vehicle.Stock_Quantity__c > 0) {
33      order.Status__c = 'Confirmed';
34      vehicle.Stock_Quantity__c -= 1;
35      ordersToUpdate.add(order);
36      vehiclesToUpdate.add(vehicle);
37  }
38  }
39  }
40
41  if (!ordersToUpdate.isEmpty()) {
42      update ordersToUpdate;
43  }
44
45  if (!vehiclesToUpdate.isEmpty()) {
46      update vehiclesToUpdate;
47  }
48  }
49  }
50
51  global void finish(Database.BatchableContext bc) {
52      System.debug('Vehicle order batch job completed.');
```

```

53  }
54  }

```



Source Code:

```

global class VehicleOrderBatch implements Database.Batchable<sObject> {
    global Database.QueryLocator start (Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c

```

'Pending'

```
]); } global void execute(Database.BatchableContext bc, List<Vehicle_Order_c>
orderList) { Set<Id> vehicleIds = new Set<Id>(); for (Vehicle_Order_c order orderList)
{ if (order.Vehicle__c != null) { vehicleIds.add(order.Vehicle__c);
} } if
(!vehicleIds.isEmpty()) {
Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
for (Vehicle_c vehicle: [SELECT Id, Stock Quantity__c FROM Vehicle_c WHERE Id IN
vehicleIds]) { vehicleStockMap.put(vehicle.Id, vehicle);
}
List<Vehicle_Order_c> ordersToUpdate = new
List<Vehicle_Order_c>(); List<Vehicle__c> vehiclesToUpdate = new
List<Vehicle__c>(); for (Vehicle_Order_c order orderList) { if
(vehicleStockMap.containsKey(order.Vehicle__c)) { Vehicle_c vehicle =
vehicleStockMap.get(order.Vehicle__c); if (vehicle.Stock_Quantity__c
> 0) { order.Status__c = 'Confirmed';
vehicle.Stock_Quantity__c -= 1;
ordersToUpdate.add(order);
vehiclesToUpdate.add(vehicle);
}
} } if
(!ordersToUpdate.isEmpty()) {
update ordersToUpdate;
```

```

} if (!vehicles ToUpdate.isEmpty())

{ update vehiclesToUpdate;

} } global void finish(Database.BatchableContext
bc) {

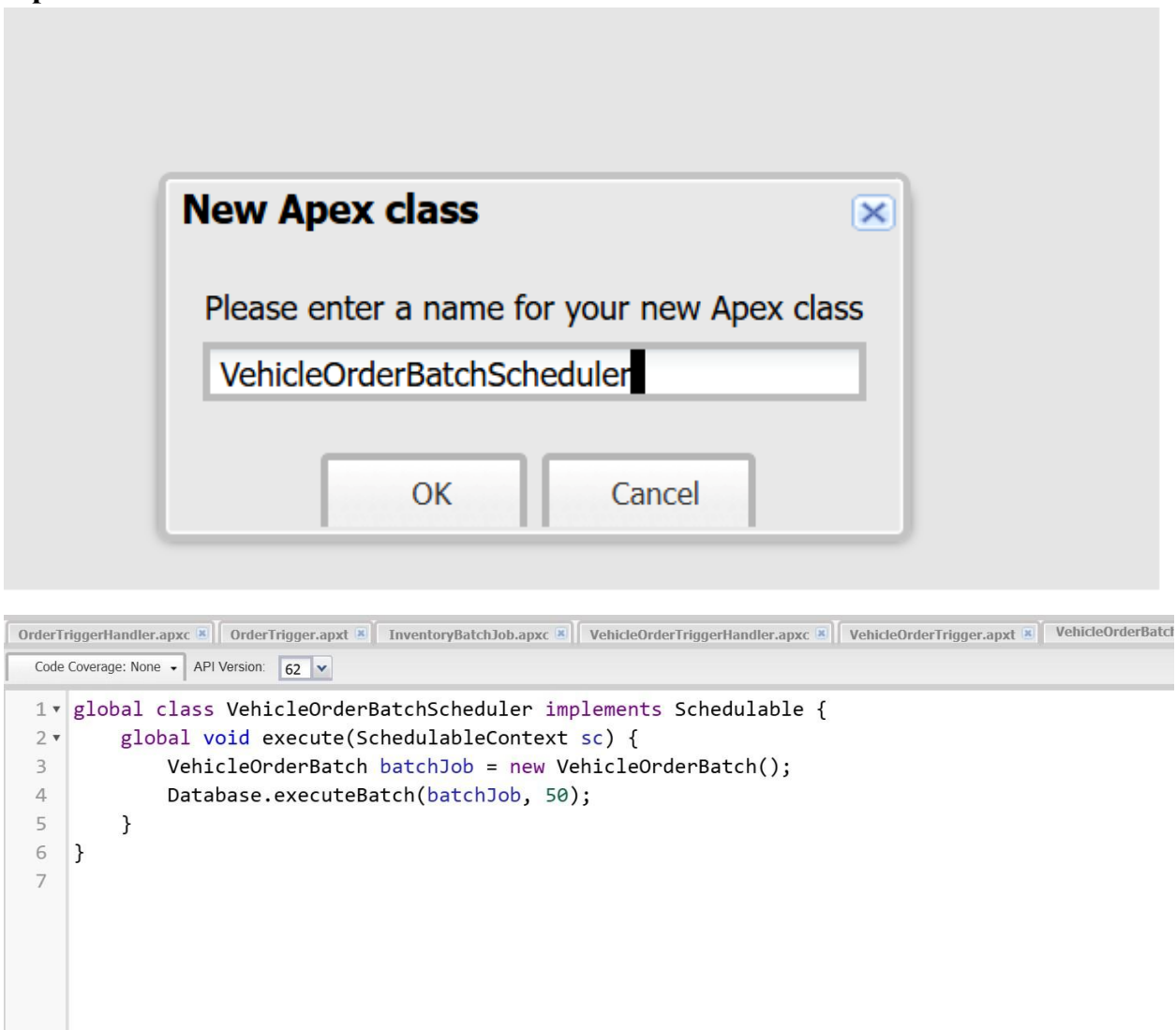
System.debug('Vehicle order batch job completed.');
```

```

}

}
```

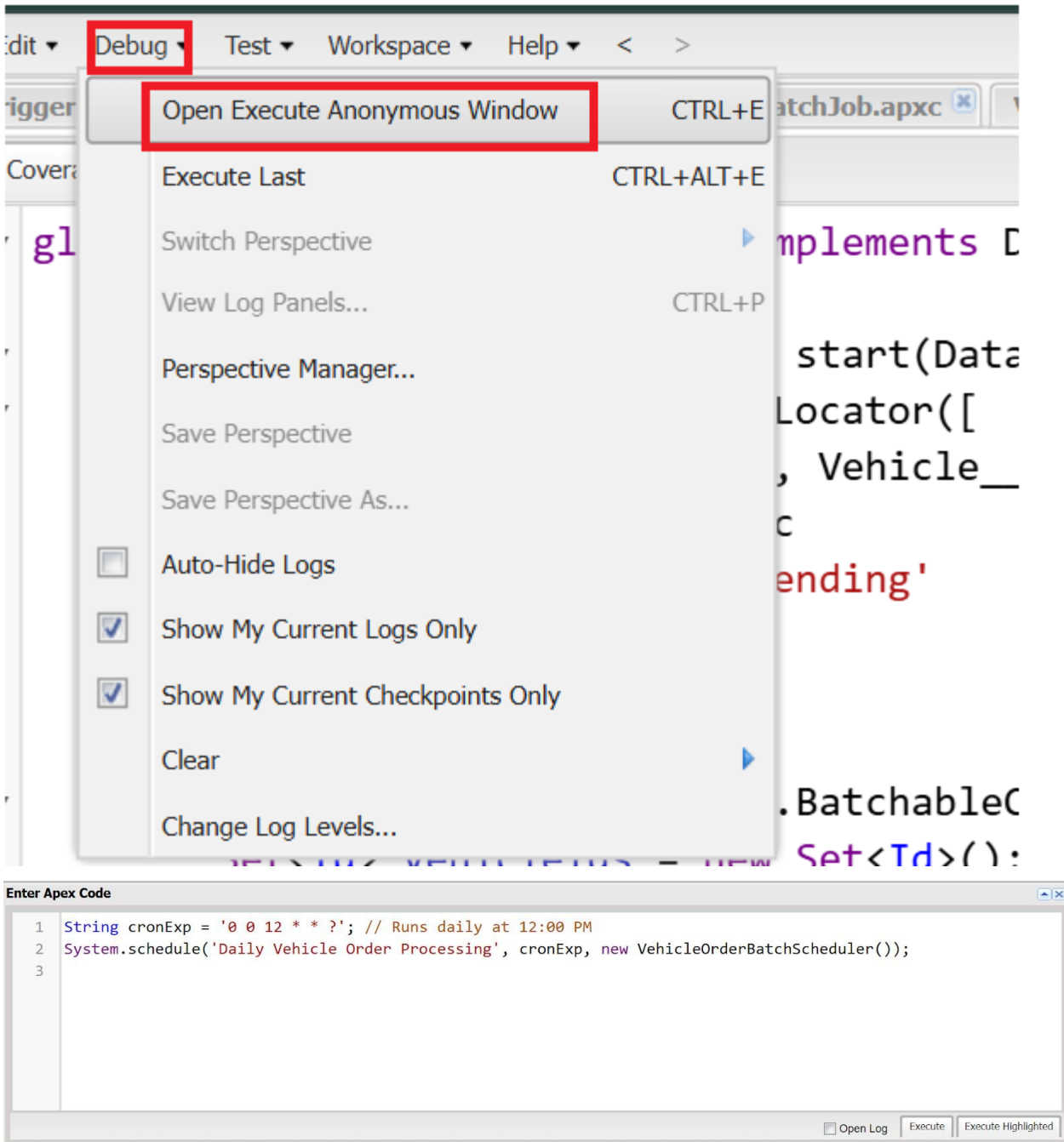
Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class



Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {  
    global void execute(SchedulableContext sc) {  
        VehicleOrderBatch batchJob = new VehicleOrderBatch();  
        Database.executeBatch(batchJob, 50); // 50 is the batch size  
    }  
}
```

Step 10 : Schedule the Batch Job To run the batch job **every night at midnight** :



Source Code :

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp, new
VehicleOrderBatchScheduler());
```

Step 11 : You Can Check where this Schedule Job is Running

Setup | Home | Object Manager

Scheduled Jobs

All Scheduled Jobs

The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.

Percentage of Scheduled Jobs Used: 2%
You have currently used 2 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: **All Scheduled Jobs** | Create New View

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Inventory Sync	Trainer, Abdul	12/02/2025, 4:24 pm	13/02/2025, 12:00 am	14/02/2025, 12:00 am	Scheduled Apex	08edM00000CH3b
Manage Del Pause Job	Daily Vehicle Order Processing	Trainer, Abdul	13/02/2025, 5:14 am	13/02/2025, 12:00 pm	14/02/2025, 12:00 pm	Scheduled Apex	08edM00000CJ99
Del	Metalytics Data Loader Job for Org - 000dM00000CJX0S	User, Integration	31/01/2025, 5:38 pm	12/02/2025, 7:42 pm	13/02/2025, 7:42 pm	Autonomous Data Loader Job	08edM00000CJntL
	Program Milestone Computation Cron Job	Process, Automated	31/01/2025, 5:38 pm	13/02/2025, 6:30 am	13/02/2025, 1:29 pm	Program Milestone Computation Cron Job	08edM00000CJntL
	Program Status Update Cron Job	Process, Automated	31/01/2025, 5:38 pm	13/02/2025, 9:31 am	13/02/2025, 6:30 pm	Program Status Update Cron Job	08edM00000CJntL

7. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate vehicle creation, dealer assignment, stock validation triggers, order status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Centralized data
- Automated workflows
- Real-time reporting

Disadvantages

- Salesforce dependency
- Requires initial configuration effort

9. Conclusion:

The Salesforce CRM implementation at WhatNext Vision Motors successfully modernizes vehicle ordering and dealership management. Automation improves efficiency, accuracy, and customer experience.

10. FUTURE SCOPE

- Mobile ordering app
- Dealer GPS integration
- SMS notifications
- AI-based demand forecasting

11. APPENDIX

- Source Code: Salesforce Configuration
- Dataset: Salesforce Objects
- GitHub / Demo: Salesforce Developer Org

Thank You